# Stable answers, active computation

A mathematical guide to the proposed architecture, the attracting-answer-set alternative, and the evidence so far

**Version 2.1 — 12 November 2026.**
Repository: NiccoGrillo/TinyRecursiveModels.
Previous document commit: `b747abb`; pilot report snapshot: `98689fb`.

## The idea in one paragraph

Keep a vector of unresolved consistency errors instead of immediately combining them into one energy gradient. A persistent workspace can mix these errors before they update the answer state. This preserves every common residual zero while allowing departure from some wrong states where the original gradient cancels. The leading question is whether learned interactions improve reasoning beyond fixed or random mixing and established nonlinear solvers. The previous energy-preconditioning architecture remains a baseline, and the attracting-answer-set alternative remains a serious branch.

## What changed in the latest critique session

1. **The critique changed the architecture.** The earlier preconditioner protects wrong stationary points as well as solutions. Residual-side cross-coupling is the stronger candidate developed in Section 1.
2. **The novelty claim became narrower.** Randomized residual mixing and Sketched Newton–Raphson are close precedents. Useful learned control must beat these methods; root preservation alone is not new.
3. **The QRR reviews changed the comparison design.** Separate fixed-compute solving from adaptive stopping, match checkpoint settings, and report failures and cap hits.
4. **There is no new trained result.** A bounded residual-mixing operator and analytic/autograd tests now exist. The complete architecture, residual training and performance claims remain untested.

This is a project specification and research record, not a paper reporting a successful new model. Both the editable source and the compiled PDF are maintained together.

# How to read this document

Start with Section 1 for the latest research session, equations, critique rounds and decision. Sections 2–4 retain the previous state-preconditioning baseline and its motivation. Section 5 supplies the proofs; Section 6 explains how a trainable model would be built. Sections 7–9 distinguish expected gains, competing ideas and the next substantial comparison. Appendix A is a self-contained treatment of attracting answer sets. The remaining appendices preserve the earlier experiments, theory, exact protocols and provenance.

**Status vocabulary.** *Observed* means reported in the committed trained-checkpoint pilot. *Proved* means derived here under explicit assumptions. *Implemented* means code exists, not that a benchmark succeeded. *Proposed* means an unbuilt or untested idea. No new complete architecture has been trained or benchmarked in this repository. The residual-mixing operator has focused analytic and automatic-differentiation checks.

**Evidence boundary.** Pilot values are transcribed from committed STATUS.md and PILOT.md; raw pilot records are not present in this documentation workspace. The current update does not claim a fresh reanalysis. Historical sections retain the November 6 reasoning and protocols under explicit archival labels; their old gates do not govern the present architecture programme.

# Contents

How to read this document . . . 2
1 Research session: preserve solutions, not every stationary point . . . 3
2 The original question and the version 2.0 baseline . . . 11
3 Mathematical vocabulary, introduced from the model . . . 13
4 The state-preconditioning baseline architecture . . . 15
5 Exactly what the architecture guarantees . . . 17
6 How to build and train the proposed model . . . 19
7 Why this architecture might help, and how it could fail . . . 21
8 QRR, related work, and the other architectural proposals . . . 23
9 Version 2.0 comparison design and living record . . . 25
A The attracting-answer-set architecture, developed fully . . . 27
B The actual machine: what TRM updates . . . 33
C Fixed points, stable answers and moving states . . . 35
D The experiment ledger: what actually exists . . . 36
E What the natural-continuation pilot tells us . . . 37
F What the perturbation experiment changed . . . 38
G Why the early diagnostic programme changed direction . . . 40
H Historical coupled-feedback insight . . . 41
I Optional historical E4 protocol, preserved in full . . . 43
J Historical E4/E5 decision rules . . . 45
K Earlier literature review: November 6 snapshot . . . 46
L Historical TRM proofs and coupled-feedback example . . . 48
M What the uncertainty numbers mean . . . 52
N Data, weights and exact provenance . . . 53
O Historical diagnostic commands and code map . . . 55
P Glossary and notation . . . 57
References and primary-source links . . . 59

# 1 Research session: preserve solutions, not every stationary point

## 12 NOVEMBER 2026 — FOUR ADVERSARIAL REVIEW ROUNDS; NEW LEADING CANDIDATE

### 1.1 The conclusion before the details

The previous proposal lets a persistent workspace reshape the gradient of one scalar energy. Its central protection is also its central weakness: *every* zero-gradient state is protected, including a wrong local minimum. Moreover, a learned history-dependent preconditioner is already a learned optimizer. That proposal remains a useful baseline, but its invariant fiber is not a sufficient novelty argument.

The stronger candidate gives the model a vector of consistency residuals rather than immediately reducing all disagreement to one gradient. A workspace mixes those residuals *before* they are sent back into the semantic state. Off-diagonal mixing allows an unresolved constraint to borrow a direction that is visible through another constraint. This can move out of some wrong local minima that neither the earlier preconditioner nor ordinary independent constraint weights can leave. It still leaves every common zero of the residuals fixed.

The new goal is therefore **solution-preserving residual mixing**, with learned memory as a hypothesis to test rather than a feature whose value is assumed. The mathematical distinction is solid. The claims that it is novel enough, that real models encounter the relevant obstruction, and that learned memory improves on cheap random mixing are still open. The critic did not approve a publication-ready method.

**What to carry forward.** Protect the states where the modeled constraints agree, rather than every state where their aggregate gradient happens to cancel. This changes the architecture's protected set. It does not guarantee that learned constraints describe correct answers or that the controller finds a solution.

### 1.2 What the supplied QRR reviews change

The user supplied three review texts in *Pasted markdown(1).md*, identifying reviewers 6QHq, 9Sdw and tDyW. We use that supplied text, not a fresh verification of the OpenReview thread or a prediction of the conference decision. The following distinctions affect our own project.

| Review issue | Assessment and consequence for this project |
|---|---|
| Architecture versus stopping rule | Strong objection. A quotient residual and a raw-state residual do not test the same notion of completion. First compare accuracy at fixed total compute without adaptive halting. Then cross architectures with the same deployable readout rule, followed by each native rule. |
| Checkpoint and coverage mismatch | Strong objection. Record source, training configuration, controller settings and precision. Count every example, including cap hits. Conditional runtime on solved or halted cases can conceal failures. |
| Symmetry versus fast quotient spectrum | Strong distinction. Exact gauge decoupling does not force rapid quotient convergence. Likewise, our common-zero protection does not imply faster or more accurate solving. |
| Non-isolated latent solutions | A valid scope question, but unique discrete answers do not imply isolated latent roots, and multiple discrete answers do not imply a continuous latent manifold. Analyze the actual state geometry and the specific theorem assumptions. |

| Review issue | Assessment and consequence for this project |
|---|---|
| Semantic collapse on gauge orbits | Equivariance plus invariant readout protects future quotient behavior, not just the current logits. “Same current output” alone would be insufficient. The remaining question is whether the imposed representational restriction harms task capacity. |
| Lack of chain-of-thought | Limits demonstrated application scope; it is not itself a mathematical defect in latent reasoning. We should not claim natural-language/process-supervision benefits from puzzle results. |
| Alleged need for labeled trajectories | Not supported by QRR’s described segment-wise prediction supervision. Reusing an answer label across segments is not a labeled reasoning trajectory. Gauge symmetry does not by itself prohibit outcome-level RL; successful RL training remains untested. |

The reviews motivate stricter comparisons, not copying every criticism as fact. In particular, time until the correct output first appears is an offline diagnostic using labels, not an executable stopping rule. A common patience rule may halt on a stable wrong answer, so compare the full accuracy–cost tradeoff and calibrate thresholds on development data.

### 1.3 Round 1: why the previous construction is too protective

Write a vector of residuals $c(q;x) \in \mathbb{R}^r$, its Jacobian $J(q) = D_q c(q;x) \in \mathbb{R}^{r\times n}$, and the scalar energy

$$E(q;x) = \tfrac{1}{2}\left\|c(q;x)\right\|^2, \qquad \nabla E(q;x) = J(q)^\top c(q;x).$$

The earlier update is

$$q' = q - \alpha M(q,w,x)J(q)^\top c(q;x).$$

At any $J^\top c = 0$, it is frozen for every workspace and every finite matrix $M$. There are two ways to obtain zero: the constraints are satisfied, $c = 0$, or their effects cancel or vanish despite $c \neq 0$. A readout can also be wrong at $c = 0$ if the residual model is badly learned. Keeping these cases separate is essential.

The limitation applies to the stated multiplicative-gradient interface, not to every learned optimizer. A generic stateful optimizer with additive momentum or exploratory proposals can move at zero current gradient. Such methods remain strong controls. They may also disturb a correct root unless their carried velocity or forcing is handled appropriately.

### 1.4 Round 2: move the workspace to the residual side

The revised update is

$$\begin{aligned} c_t &= c_\theta(q_t;x), \qquad J_t = D_q c_\theta(q_t;x), \\ q_{t+1} &= q_t - \alpha J_t^\top W_\theta(q_t,w_t,x)c_t, \\ w_{t+1} &= B_\theta(w_t,q_t,c_t,x), \qquad \hat{y}_t = D_\theta(q_t). \end{aligned} \tag{1}$$

Here $W \in \mathbb{R}^{r\times r}$ acts on residual coordinates, whereas the old $M \in \mathbb{R}^{n\times n}$ acts on semantic coordinates. The order is consequential: in general $J^\top W c$ cannot be recovered by multiplying the already-zero vector $J^\top c$.

A positive diagonal $W$ simply changes the weight of each squared residual. This already breaks some cancellation equilibria, but constraint reweighting has extensive prior art, including Breakout and analog SAT dynamics [23, 24]. It also fails whenever every individual vector $c_i \nabla c_i$ is zero. The next round therefore targets interactions between residuals, not just changing their separate weights.

### 1.5 Round 3: off-diagonal mixing has a stricter separation

Consider one scalar semantic state $q$ and two residuals

$$c_1(q) = q^2 - 1, \qquad c_2(q) = 2q(q-1). \tag{2}$$

Their common zero is $q_* = 1$. At the wrong state $q = 0$,

$$c(0) = \begin{pmatrix} -1 \\ 0 \end{pmatrix}, \quad J(0) = \begin{pmatrix} 0 \\ -2 \end{pmatrix}, \quad E'(0) = 0, \quad E''(0) = 2 > 0.$$

Thus $q = 0$ is a strict local minimum of the unweighted energy, not merely a saddle. The first residual is unsatisfied but locally flat; the second is satisfied but has a nonzero derivative. Both individual squared-residual gradients are zero. Every diagonal residual weighting and every old state-side preconditioner therefore proposes zero movement there.

Now choose

$$W = \begin{pmatrix} 1 & -\eta \\ -\eta & 1 \end{pmatrix}, \quad 0 < \eta < 1.$$

Its eigenvalues are $1 - \eta$ and $1 + \eta$, so it is positive definite. Direct multiplication gives

$$J(0)^\top W c(0) = -2\eta, \qquad q' = 2\alpha\eta > 0.$$

The unresolved first residual is transmitted through the second residual's derivative. The common solution $q = 1$ remains fixed because $c(1) = 0$.

This proves a difference between update classes. It does not prove that a rollout reaches one, that this mechanism occurs in trained TRM, or that learning a workspace is needed: a fixed $W$ already moves in this example. Those are precisely the rival explanations the experiment must address.

### 1.6 A bounded low-rank implementation

Use

$$W = mI + \frac{bUU^\top}{1 + \|U\|_F^2}, \qquad m > 0, \quad 0 < b \le b_{\max}, \quad U \in \mathbb{R}^{r \times k}. \tag{3}$$

Then $mI \preceq W \preceq (m + b_{\max})I$. Off-diagonal entries can be negative even though $UU^\top$ is positive semidefinite. Low rank does not mean diagonal: a rank-one vector with mixed signs already couples residuals. Compute

$$v = Wc = mc + \frac{bU(U^\top c)}{1 + \|U\|_F^2}, \qquad f = J^\top v,$$

using $O(rk)$ arithmetic for mixing and one vector-Jacobian product (VJP) for the force. The residual network and its derivatives still dominate cost in many settings; this is not a free operation.

The VJP must calculate $J^\top v$ with $v$ as its supplied cotangent. Taking the total derivative of $\frac{1}{2} c^\top W(q, w) c$ would add derivatives of $W$ and implement a different update. Conversely, permanently detaching $v$ would lose training gradients to the mixing controller. The correct construction treats the cotangent as fixed in the inner VJP while retaining its graph for differentiation of the completed update during outer training. The operator prototype in `experiments/residual_reasoning/operators.py` checks that distinction with finite differences. It is not a complete trained reasoner; the focused tests are in `tests/residual_reasoning/test_operators.py`.

## 1.7 Which states are protected for every workspace choice?

**Proposition 3 (common-root family).** Fix $q, x$, and allow $W$ to range over an open neighborhood of positive-definite matrices. Then

$$J^\top W c = 0 \text{ for every admissible } W \quad \iff \quad c = 0 \text{ or } J = 0.$$

**Proof.** The reverse implication is immediate. If $c \neq 0$, variations of $W$ around an interior point include small symmetric matrices $S$. For every $v \in \mathbb{R}^r$, the symmetric matrix

$$S = \frac{vc^\top + cv^\top}{\|c\|^2} - \frac{v^\top c}{\|c\|^4} cc^\top$$

satisfies $Sc = v$. Subtracting the equations for nearby admissible matrices yields $J^\top Sc = 0$ for every $v$, hence $J = 0$. ☐

At an ordinary cancellation root $J^\top c = 0$ with $c \neq 0, J \neq 0$, even the rank-one family in (3) has an escaping choice. Pick $a$ with $Ja \neq 0$ and set $u = c + Ja$. Then $u^\top c = \|c\|^2$ and $J^\top u = J^\top J a \neq 0$. The low-rank correction therefore gives a nonzero force.

The quantifier matters: this is an intersection over *possible* workspace-controlled matrices. A trained deterministic controller can select bad matrices, converge to an unhelpful weight pattern, or enter a wrong cycle. The proposition does not say the trained architecture has no incorrect equilibria. If $J = 0$ with $c \neq 0$, all such first-order residual mixing is blind. Extra search mechanisms would be required.

## 1.8 Local attraction without requiring the original energy to decrease

Escaping a strict local minimum of $E$ necessarily increases $E$ for sufficiently small nonzero steps. An Armijo gate on that same unweighted energy would reject the very behavior we seek. The previous global common-energy argument therefore does **not** transfer to this proposal.

There is instead a local common-distance argument near a regular zero. Suppose $c(q_*) = 0$, $c$ is twice continuously differentiable nearby, $J_* = J(q_*)$ has full column rank with smallest singular value $\sigma > 0$, and $mI \preceq W \preceq \bar{m}I$ uniformly. Put $e = q - q_*$ and $f_W(q) = J(q)^\top W c(q)$. Taylor expansion gives, uniformly over those $W$,

$$f_W(q) = J_*^\top W J_* e + O(\|e\|^2).$$

In a sufficiently small ball there is a finite $K$ such that

$$e^\top f_W(q) \geq \tfrac{1}{2} m\sigma^2 \|e\|^2, \qquad \|f_W(q)\| \leq K \|e\|.$$

Consequently,

$$\|e - \alpha f_W(q)\|^2 \leq (1 - \alpha m \sigma^2 + \alpha^2 K^2) \|e\|^2. \tag{4}$$

Choosing a sufficiently small fixed $0 < \alpha < m\sigma^2/K^2$ gives a contraction factor below one and makes the ball invariant. The bound is common to all admissible $W$, so arbitrary workspace switching is allowed locally. No bound on $D_q W$ is needed for this pointwise argument. The constants and the neighborhood are conditional mathematical quantities, not certified values for an untrained neural model.

For a smooth zero manifold $\mathcal{M} = \{q : c(q) = 0\}$, a parallel argument works in a uniform regular tube around a compact component, or on a regular tube in whose geometric domain the iterates are separately assumed to remain. The tube must have a unique nearest projection $p$, uniform derivative bounds, and a uniform lower singular-value bound for $J(p)$ restricted to normal directions. The normal error $e = q - p$ obeys the same estimates. Since

$$\operatorname{dist}(q - \alpha f_W(q), \mathcal{M}) \leq \| q - \alpha f_W(q) - p \| ,$$

small-step distance contraction follows without assuming the manifold is a single point. This requires regular geometry; singular or rank-changing zero sets are excluded. Readout correctness and a robust margin neighborhood remain separate assumptions.

On the zero set the semantic update stops; only workspace motion persists. A model that also moves tangentially in semantic coordinates still needs the additional constructions in Appendix A. This proposal does not silently solve that harder problem.

## 1.9 Round 4: the random-mixing and change-of-basis objections

The critic's strongest cheap control is fresh random rank-one mixing. At a cancellation state with $c \neq 0, J \neq 0$, an absolutely continuous, full-dimensional random $u$ has $(u^\top c) J^\top u \neq 0$ almost surely. Therefore random mixing already produces a nonzero step, without learned memory. This is only departure from the current point, not guaranteed escape from a basin or eventual solution.

For an unnormalized Gaussian $u \sim \mathcal{N}(0, I)$, the corresponding force $h = J^\top u u^\top c$ satisfies

$$\mathbb{E}[h] = J^\top c, \qquad \mathbb{E}\, \|h\|^2 = \|c\|^2 \, \|J\|_F^2 + 2 \left\|J^\top c\right\|^2 .$$

The second identity follows from Gaussian fourth moments. At aggregate cancellation it has zero mean but nonzero variance. At $c = 0$ it vanishes exactly. This is structured, residual-dependent exploration. The bounded normalized operator has a different expectation and second moment, so these displayed formulas must not be copied onto it unchanged.

For fixed $u$, $h$ is the gradient of the randomly projected residual loss $\frac{1}{2}(u^\top c)^2$. This connects the mechanism to randomized least-squares sketching, which is established literature [25]. Likewise, a fixed SPD $W = L^\top L$ can be absorbed into a new residual basis $\tilde{c} = Lc$. Hence a fixed mixing matrix is not inherently a new expressive neural architecture if the residual head is freely learnable.

A closer predecessor is *Sketched Newton–Raphson* (SNR) [26]. Its Section 3, equations (3.1)–(3.5), formulates residual losses with a sketch-dependent metric frozen for the inner derivative, then updates that metric at the next iterate. In our Jacobian convention its force has the form $J^\top H_S c$, with $H_S = S(S^\top J J^\top S)^\dagger S^\top$. Its analysis explicitly uses the fact that every residual zero makes each stochastic gradient vanish. SNR's specific metric is positive semidefinite and may involve a pseudoinverse; our proposed bounded learned SPD controller is different, but the general residual-space construction and root preservation are established. SNR, including a documented regularized variant where needed, is therefore a direct solver control, not merely a related-work citation.

Persistent constraint memory is also not enough by itself: the recent *Lagrangian Brain* manuscript [27], Appendix F.3, updates constraint multipliers while changing discrete candidate states. Its Sudoku protocol allows unrestricted thinking time; its headline result must not be treated as an equal-cost comparison. We have inspected that update description, not reproduced its results or verified its wider claims.

Memory-dependent mixing could adapt which residual combinations matter as the search evolves. That is the remaining hypothesis. It must beat fixed full mixing, stateless full mixing, random mixing and a generic factor-aware learned optimizer. We should drop the memory component if it does not earn its extra cost. Even if random mixing works well, its optimization interpretation must be acknowledged rather than advertised as a wholly new principle.

Residual coordinates also have conventions: replacing $c$ with $Sc$ for nonsingular $S$ preserves its zero set but changes its dynamics unless $W$ transforms consistently. Factor permutations can be handled by a shared equivariant controller; arbitrary sign and scale changes require explicit design or a documented fixed convention. A freely rescaled learned head can otherwise manufacture the apparent effect. Normalize and pin residual units for the controlled comparison, then separately test a freely learned head. Do not claim basis-independent geometry without proving it.

### 1.10 Concrete first architecture: grounded constraints and learned coordination

The critique exposed a major avoidable ambiguity: a learned residual can collapse to zero or encode wrong answers. The first full architecture should therefore use **known task residuals and a learned factor controller**. This is a neuro-symbolic reasoner, not a general-purpose latent language model. The residuals and verifier access must be identical for all controlled baselines.

For Sudoku, let $p_{i,d} \in \mathbb{R}$ represent cell $i$ choosing digit $d$. Include all of the following residuals, with fixed documented scales:

$$c^{\text{bin}}_{i,d} = p_{i,d}(1 - p_{i,d}), \qquad c^{\text{cell}}_{i} = \sum_{d} p_{i,d} - 1,$$

$$c^{\text{unit}}_{U,d} = \sum_{i \in U} p_{i,d} - 1, \qquad c^{\text{clue}}_{i} = p_{i,d_i} - 1.$$

Here $U$ ranges over every row, column and box, and $d_i$ is the observed clue digit, not the unknown answer. All Boolean residuals must be included. A common zero forces every entry to be zero or one, one digit per cell, one occurrence of each digit per unit, and preservation of clues. Therefore every common zero is a valid one-hot solution, including alternative valid solutions when the puzzle is not unique. This removes residual-collapse ambiguity for the first study.

At any such zero, the Jacobian rows of $c^{\text{bin}}$ form a diagonal matrix with entries $1 - 2p_{i,d} \in \{-1, 1\}$. Thus the full residual Jacobian has full column rank: with unit Boolean scaling, $J^\top J \succeq I$. The regular-root premise of the local attraction argument is therefore structurally satisfied. This does not supply a global basin or a universal step size; polynomial derivatives outside the Boolean region can be large.

Use a bipartite variable–factor network with shared updates. Variable nodes carry $p$, input clues and structural type information; factor nodes carry their residual, type and bounded memory $w_a$. A factor's output row $U_a \in \mathbb{R}^k$ specifies its participation in the low-rank residual mixing. Message passing respects variable/factor relabeling; ordering conventions, digit symmetries and signed incidence must be implemented explicitly rather than merely asserted. Both current residuals and memory may inform $U$. Read out a Sudoku by argmax across each cell's digits and verify all constraints. Keep the same initial-state distribution and graph information across controls.

The mathematical specification initially uses unconstrained real $p$, without sigmoid saturation or projection, so Boolean zeros occur at finite states and the stated VJP update applies literally. It does not guarantee that iterates stay in $[0, 1]$ or remain bounded globally. Count numerical divergence and invalid output as failures. If clipping, projection or an acceptance policy is added, document it as a changed update, apply it comparably to controls and recheck the local claims.

A second factor-graph family can use satisfiable 3SAT: Boolean residuals $p_i(1 - p_i)$ and one clause residual equal to the product of its literal-false factors, $(1 - p_i)$ for a positive literal and $p_i$ for a negative literal. Common zeros are exactly satisfying assignments and the same Boolean Jacobian argument applies. This is witness finding, not an UNSAT certificate. At an unsatisfying Boolean vertex, violated-clause gradients agree in sign for any shared variable, so the aggregate cancellation mechanism of interest concerns fractional search states, not arbitrary wrong Boolean vertices.

This grounded setup is the implementation target, with Sudoku providing continuity and a second family testing whether coordination transfers. Exact sizes, training budget and datasets remain to be frozen before a training launch. A fully learned latent residual head remains a later branch: it would require explicit noncollapse and answer-alignment training and cannot inherit the grounded correctness theorem. The present operator prototype does not yet implement the factor network or task residual modules.

### 1.11 The substantial experiment that could justify the project

The central comparison is a shared-residual search study with the following mechanisms: state-side preconditioning; diagonal adaptive weights; fixed off-diagonal mixing; stateless learned mixing; recurrent learned mixing; fresh random low-rank mixing; Sketched Newton–Raphson, and a strong factor-aware learned optimizer or solver. Published energy reasoners and QRR provide external references with clearly separated reproduction settings. Match total inference cost, account for training resources and parameter capacity, and do not handicap baseline access to residual information.

First run fixed total-compute budgets with no adaptive stopping. Primary outcomes are exact validity/accuracy across all instances, cost curves, and task-size/depth generalization over multiple training seeds. Use a task family with room to improve and meaningful constraint interactions; avoid making a near-ceiling Sudoku percentage the whole paper. Cases with multiple valid answers require validity-aware scoring, not rejection of alternative correct solutions because only one label was stored.

Within the same study, identify states with large $\|c\|$ but small $\left\|J^\top c\right\|$, then perform matched continuation interventions: change only $W$, reset workspace, substitute random mixing, and compare the resulting valid-solve rate at equal remaining cost. Numerical thresholds and cohorts must be fixed on development data. Small gradient is not evidence of cancellation unless residual size and derivative information support that diagnosis. This is a mechanistic analysis embedded in the substantial run, not a gate requiring another series of tiny pilots.

Only afterwards compare a shared deployable stopping rule across models and native stopping rules separately. Calibrate on development data, count rule evaluation cost, include cap hits and all failed examples, and report accuracy–coverage–runtime curves. Output stabilization and first correctness can be offline references, never oracle deployment policies. This directly addresses the strongest QRR review confound.

A convincing result would show that learned off-diagonal interactions improve valid solving and cost/generalization beyond random mixing and generic factor-aware optimization, with evidence connecting the gain to unresolved residual interactions. If random mixing matches the controller, retain the simpler method and narrow the claim. If a freely learned static residual basis erases the gain, reject the dynamic-mixing explanation. If benefits appear only through a more favorable stopping test, reject the faster-computation claim.

### 1.12 Critique ledger and current status

| Round | Objection | Revision or consequence |
|---|---|---|
| 1 | V2 is a learned optimizer and protects wrong roots. | Retain as baseline; examine residual-side control. |
| 2 | Diagonal weighting is old and can remain blind. | Analyze off-diagonal interactions and explicit strict-minimum separation. |
| 3 | Available escape does not guarantee a learned controller escapes; global descent conflicts with escape. | State family quantifiers; replace global descent claim with conditional local root/set attraction. |
| 4 | Random mixing and a learned static basis may explain the whole gain. | Make both strong controls; learning and memory remain contingent. |

**Solid now:** an exact distinction between update classes, a bounded operator specification, conditional local attraction, and a falsifiable architectural comparison. These are not claimed as new general optimization theorems. **Not solid yet:** the full trained factor controller, practical gains, empirical mechanism frequency, or publication-level novelty. The correct stopping point for this research session is a clear hypothesis and implemented mathematical core, not a critic pressured into declaring success.

The remaining main sections preserve the version 2.0 state-preconditioning baseline, its proofs and training discussion. They are useful comparison material, but this section supersedes their claim to be the sole leading architectural direction. The attracting-set appendix and historical pilot evidence remain intact.

## 2 The original question and the version 2.0 baseline

A recurrent reasoner repeatedly applies a learned computation to the same problem. If its entire state must converge to one vector, all internal degrees of freedom eventually stop changing. But an answer is only a small part of the information stored in the network. It is natural to ask whether answer stability can coexist with continuing internal motion.

There are three different claims here. First, many hidden states can decode to the same answer; this follows from the readout being many-to-one. Second, those states could form an invariant, attracting set under the model's dynamics. That requires architectural or dynamical conditions. Third, allowing such a set could make reasoning faster or more accurate. That is an empirical hypothesis, not a consequence of the first two claims.

Our initial project tested frozen TRM checkpoints and developed a schedule-dependent feedback example. The pilot found recovery without observed retention failure. The example established that update timing can change stability while preserving a root. These results remain useful, but neither requires postponing architectural research until a particular failure is observed. At the user's request, version 2.0 prioritized a new model directly. The following baseline motivation is preserved; Section 1 records the subsequent critique and revised leading candidate.

### 2.1 The central tension: harmless motion versus useful motion

Suppose a hidden variable can never affect the answer, now or at any future time. It is safe, but it cannot help solve the problem. Conversely, if a workspace can change the semantic state without restriction, it can disturb an answer after solving it. The proposed architecture resolves this tension *conditionally*: workspace motion matters away from stationarity, and its semantic influence vanishes at stationarity.

This is different from demanding that an arbitrary recurrent Transformer contract everywhere. It is also different from adding an oscillator that the answer head ignores. The workspace must alter the actual search trajectory before completion; otherwise its persistent motion is decorative.

**What to carry forward.** The project hypothesis is that history-dependent search geometry can improve finite-budget reasoning while a structurally protected semantic interface makes stationary answers robust to continued workspace evolution. Both usefulness and task correctness must be learned and measured.

## 2.2 What is retained from each previous direction

| Idea | Current role |
|---|---|
| Full-state fixed points | A baseline and mathematical tool; not assumed to be unique or inherently bad. |
| Attracting answer sets | A serious alternative architecture, developed in Appendix A. |
| Transient amplification | A warning that stable eigenvalues alone do not bound finite-time disturbances. |
| TRM coupled feedback | A proved artificial example and optional diagnostic programme; no longer a prerequisite. |
| QRR quotient dynamics | Inspiration for separating answer-relevant and answer-irrelevant motion; not a proof that every useful mode should be quotiented out. |
| Persistent workspace | Version 2.0 proposal: affect search through a bounded positive-definite metric and lose influence at energy stationarity. |

# 3 Mathematical vocabulary, introduced from the model

## 3.1 States, maps, readouts and fixed points

Fix an input $x$, such as a Sudoku puzzle. A discrete dynamical system is an update $s_{t+1} = F_x(s_t)$ on a state $s_t \in \mathbb{R}^n$. A fixed point satisfies $F_x(s_*) = s_*$. There can be many fixed points, including incorrect ones. An invariant set $\mathcal{M}$ satisfies $F_x(\mathcal{M}) \subseteq \mathcal{M}$; individual points may move within it. Attraction means nearby trajectories approach the set, measured by

$$\operatorname{dist}(s, \mathcal{M}) = \inf_{u \in \mathcal{M}} \|s - u\| .$$

The decoder consists of real-valued logits $R(s)$ followed by a discrete choice $D(s)$, usually an argmax per output token. Equal answers do not require equal logits or equal states. A readout fiber $\{s : R(s) = r\}$ is simply a level set of the readout. It is not automatically invariant under $F_x$.

## 3.2 Gradient, Hessian and the geometry of descent

An energy $E(q; x) \in \mathbb{R}$ assigns one scalar to each semantic state $q \in \mathbb{R}^n$. It is learned; it is not automatically the task error. Its gradient $g = \nabla_q E$ collects first derivatives and points in the direction of steepest local increase in Euclidean geometry. The negative gradient is a descent direction because

$$\frac{d}{d\alpha} E(q - \alpha g; x)\bigg|_{\alpha=0} = -\|g\|^2 .$$

A stationary point has $g = 0$. It can be a minimum, saddle, maximum or part of a flat set. The Hessian $H = \nabla_q^2 E$ is the matrix of second derivatives. Near a nondegenerate minimum it describes curvature: some directions can be much steeper than others. Gradient descent can move slowly along a narrow curved valley; a preconditioner changes the relative scaling and mixing of directions.

A symmetric matrix $M$ is positive definite if $v^\top M v > 0$ for every nonzero $v$. The notation $mI \preceq M \preceq \bar{m} I$ means

$$m\|v\|^2 \le v^\top M v \le \bar{m}\|v\|^2 \quad \text{for every } v.$$

Its eigenvalues lie between $m$ and $\bar{m}$. Updating along $-Mg$ is still locally downhill since $g^\top(-Mg) < 0$ when $g \neq 0$. Here $M$ is an inverse metric or preconditioner: it changes *how* the energy is optimized, without changing its stationary set when $M$ is invertible.

## 3.3 Smoothness, Lyapunov functions and convergence

An energy is $\beta$-smooth on a region if its gradient is Lipschitz there: $\|\nabla E(u) - \nabla E(v)\| \le \beta\|u - v\|$. Along a segment inside that region this implies

$$E(q + d) \le E(q) + g^\top d + \tfrac{\beta}{2}\|d\|^2 .$$

This is the descent lemma. A Lyapunov function is a scalar that does not increase along trajectories and measures progress toward a target. It need not be a physical energy. Having a common Lyapunov function for every workspace state is stronger than checking that several separate linearizations look stable.

A Polyak–Łojasiewicz (PL) inequality is

$$\tfrac{1}{2}\|\nabla E(q)\|^2 \ge \mu(E(q) - E_*), \qquad \mu > 0.$$

It says that nonzero energy error cannot coexist with an arbitrarily tiny gradient in the region under consideration. It can imply geometric decay of the energy error without convexity. It does not say that there is only one minimizer or that the readout is correct. Strong convexity is a stricter condition; around its unique minimizer it supplies $E(q) - E_* \geq \frac{\mu}{2} \left\| q - q_* \right\|^2$, allowing an energy bound to imply a distance bound.

### 3.4 Jacobians and why eigenvalues are not the whole story

The Jacobian $DF(s)$ is the linear map describing how a small state perturbation changes after one update. At a fixed point, eigenvalues strictly inside the unit disk imply local asymptotic stability for a smooth discrete map. They do not exclude large transient amplification for a nonnormal matrix. Along a moving workspace trajectory the relevant linearization varies with time, and products of matrices matter. Individually stable matrices need not yield a stable switched system. Our descent argument below uses one energy shared by all workspace updates, rather than relying on independent eigenvalue checks.

### 3.5 What "manifold" and "quotient" mean here

A smooth manifold is a set that locally resembles Euclidean space of a smaller dimension. Its tangent directions stay in the set to first order; normal directions leave it. A circle has one tangent and one normal direction at each point. A quotient identifies states deemed equivalent: if a group $G$ acts on states, the orbit $G \cdot s$ is the collection of transformations of $s$, and $[s]$ denotes its equivalence class. Gauge directions are tangent to such orbits. A quotient readout ignores the chosen group transformations. These definitions describe geometry, not correctness. Appendix A develops the geometry without assuming a group action.

# 4 The state-preconditioning baseline architecture

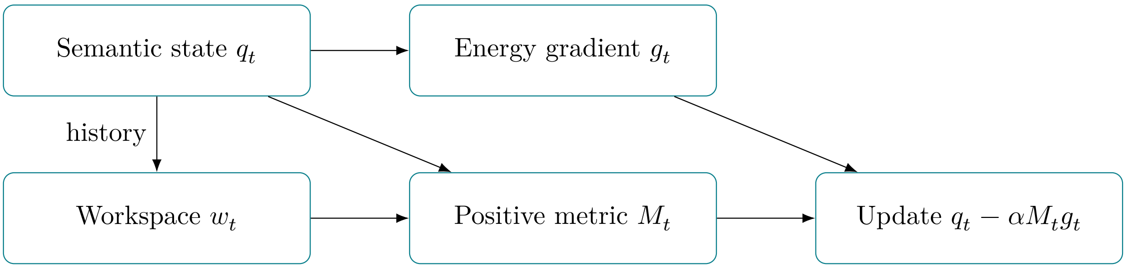
**V2 BASELINE — current leading candidate is in Section 1; no trained evaluation**

## 4.1 Two states and one shared objective

Let $q_t \in \mathbb{R}^{S\times d_q}$ be the semantic state, $w_t \in \mathcal{W} \subseteq \mathbb{R}^{S\times d_w}$ the workspace, and $x$ the fixed input. Matrices below act on flattened vectors; token structure is retained in implementation. Define

$$\begin{aligned} g_t &= \nabla_q E_\theta(q_t; x), && (5)\\ q_{t+1} &= q_t - \alpha_t M_\theta(q_t, w_t, x) g_t, && (6)\\ w_{t+1} &= B_\theta(w_t, q_t, g_t, x), && (7)\\ \ell_t &= R_\theta(q_t), \qquad \hat{y}_t = D_\theta(q_t) = \operatorname{argmax} \ell_t. && (8) \end{aligned}$$

The updates are simultaneous: both use the old $q_t, w_t$. A sequential variant would need its own specification. The energy and readout do **not** take $w_t$ as an argument. Workspace enters the semantic update only through $M$, and optionally a strictly bounded step size. The core theory below uses a fixed $\alpha > 0$ to keep assumptions transparent.



The figure suppresses the fixed input, the recurrent update of $w$, and the readout for clarity. The important restriction is the multiplicative gradient interface: no independent workspace-to-answer additive term is allowed.

## 4.2 A concrete bounded positive-definite metric

For one vector block, let $U_\theta(q, w, x) \in \mathbb{R}^{d\times r}$, with small rank $r$, and set

$$M = m_0 I + \frac{bUU^\top}{1 + \|U\|_F^2}, \qquad m_0 > 0, \quad 0 \le b \le b_{\max}. \tag{9}$$

The Frobenius norm squares and sums all entries. Since $UU^\top$ is positive semidefinite and $\|UU^\top\|_{\mathrm{op}} \le \|U\|_F^2$,

$$m_0 I \preceq M \preceq (m_0 + b_{\max}) I.$$

The denominator prevents the learned matrix from making the largest eigenvalue arbitrarily large. $m_0$ and $b_{\max}$ must remain fixed or explicitly bounded; unconstrained learned scales would invalidate this bound. Apply $M$ without forming a dense matrix:

$$Mg = m_0 g + \frac{bU(U^\top g)}{1 + \|U\|_F^2}.$$

A tokenwise block-diagonal metric costs $O(Sd_q r)$ for this multiplication. It mixes channels within each token; it is not a full arbitrary cross-token metric. Cross-token context can still enter through the attention-based energy and the network predicting each $U$. Global low-rank factors are a separate capacity/cost choice. A bounded positive diagonal plus low-rank term is an extension, with its own minimum and maximum eigenvalue accounting.

### 4.3 A bounded workspace with meaningful memory

One valid implementation is $B_\theta = \tanh A_\theta(w, q, g, x)$, with tanh applied to the entire output. Then $\mathcal{W} = [-1, 1]^{Sd_w}$ is forward invariant. A gated convex mixture of a bounded previous state and a bounded proposal also works. An unrestricted residual $w + \tanh A_\theta$ does *not* guarantee boundedness.

The workspace may store recent gradients, conflicting constraints, or which directions were ineffective. This is a proposed learned function, not a hand-proved semantic interpretation of its channels. It can keep evolving, become periodic, or converge; the architecture does not force oscillation. Attention and ordinary real-valued MLPs can implement $A_\theta$ and the metric predictor. For differentiable energy training, choose smooth energy activations and avoid hard discontinuities in the gradient path.

### 4.4 What remains fixed and what is allowed to move

At an energy stationary point $q_*$, the full set $\{q_*\} \times \mathcal{W}$ is invariant. The workspace may change while the semantic state stays exactly fixed. This is a product fiber: a simple solution set in the full state space. If several distinct $q_*$ decode correctly, there are several such fibers. The architecture does not automatically discover a curved manifold of equivalent semantic representations; that broader possibility is the appendix alternative.

It also does not automatically represent several different valid answers at once. A set of latent representations of *one* answer and a set of distinct task solutions are different objects. Multi-solution coverage would require a sampling or diversity objective and evaluation beyond ordinary single-label supervision.

**What to carry forward.** We retain stationarity in the semantic coordinates, but remove the requirement that all memory become stationary. This is a structured relaxation of full-state fixed-point reasoning, not a claim that all fixed-point constraints disappear.

# 5 Exactly what the architecture guarantees

## 5.1 Stationary fibers and vanishing workspace influence

**Proposition 1.** *Fix $x$. Suppose $E$ is differentiable, $\nabla E(q_*; x) = 0$, and $B$ maps $\mathcal{W}$ into itself. Under equations (5)–(7), $\{q_*\} \times \mathcal{W}$ is invariant, and $D(q_t)$ is constant for trajectories starting there.*

**Proof.** For every $w \in \mathcal{W}$, $q' = q_* - \alpha M(q_*, w, x)0 = q_*$. The next workspace lies in $\mathcal{W}$. Apply this repeatedly. The decoder only sees the unchanged $q_*$. No attraction or correctness assumption has been used. ☐

Where derivatives exist,

$$D_w q' = -\alpha\,(D_w M)g = 0 \quad \text{at } q_*.$$

If the step size is also differentiable and workspace-dependent, its additional derivative is multiplied by $g$ and also vanishes at a root. Away from a root, that derivative must be included in sensitivity bounds.

For fixed $\alpha, q, x$, if $M$ is $L_M$-Lipschitz in $w$ in operator norm, then

$$\|q'(q, w + \delta w) - q'(q, w)\| \le \alpha L_M \|g(q)\| \|\delta w\|. \tag{10}$$

This shows gradual one-step decoupling as the gradient shrinks. It is not a multi-step or cross-basin guarantee. A small one-step discrepancy can still change which basin is reached later.

At a stationary semantic state the full local derivative has block form

$$DF(q_*, w) = \begin{pmatrix} I - \alpha M_* H_* & 0 \\ C_* & D_* \end{pmatrix}.$$

The terms involving $D_q M$ vanish because $g = 0$. $C_*$ and $D_*$ come from the workspace update, including its dependence on $g$. If $w$ moves, this is a time-varying sequence of matrices, not one Jacobian at a full-state fixed point. We do not infer global stability or normal hyperbolicity from this block form alone.

## 5.2 A common descent inequality despite changing workspace

**Proposition 2.** *Suppose $E$ is $\beta$-smooth along each actual update segment and $mI \preceq M_t \preceq \bar{m}I$ uniformly. Then*

$$\begin{aligned} E(q_{t+1}) &\le E(q_t) - \alpha g_t^\top M_t g_t + \tfrac{\beta\alpha^2}{2} \|M_t g_t\|^2 \\ &\le E(q_t) - \alpha\left(1 - \tfrac{\alpha\beta\bar{m}}{2}\right) g_t^\top M_t g_t. \end{aligned} \tag{11}$$

**Proof.** Substitute $d = -\alpha M_t g_t$ into the descent lemma. Diagonalizing symmetric $M_t$ gives $M_t^2 \preceq \bar{m} M_t$, hence $\|M_t g_t\|^2 \le \bar{m} g_t^\top M_t g_t$. ☐

Thus $0 < \alpha < 2/(\beta\bar{m})$ makes the energy strictly decrease whenever $g_t \neq 0$. This holds for any workspace trajectory respecting the metric bounds. The same scalar energy is a common progress measure, which avoids relying on stability of independently switched matrices.

If $E$ is bounded below, the fixed step size satisfies the strict bound, and all segments remain in the smoothness region, summing (11) yields $\sum_t \|g_t\|^2 < \infty$, hence $\|g_t\| \to 0$. This does not by itself prove convergence of $q_t$, boundedness of $q_t$, global minimization, or correctness. For example, an energy can decrease along a trajectory escaping to infinity unless further assumptions prevent this. Compact sublevel sets are one way to obtain bounded trajectories.

## 5.3 What a PL inequality adds

Assume additionally that the iterates remain in a region satisfying the PL inequality with reference minimum $E_*$. Combining it with (11) gives

$$E(q_{t+1}) - E_* \le \left[1 - 2\alpha m\mu\left(1 - \tfrac{\alpha\beta\bar{m}}{2}\right)\right](E(q_t) - E_*).$$

For $\alpha \le 1/(\beta\bar{m})$ a simpler bound is

$$E(q_t) - E_* \le (1 - \alpha m\mu)^t(E(q_0) - E_*).$$

This is geometric *energy-gap* decay. A local hypothesis is only useful while the trajectory stays in its region; stating local PL does not establish that invariance. Under a compatible strong-convexity bound around a unique $q_*$, one further gets

$$\|q_t - q_*\|^2 \le \frac{2}{\mu}(E(q_t) - E_*).$$

Without such a distance/error bound, do not replace energy convergence with convergence to a particular semantic point.

## 5.4 From semantic convergence to a stable discrete answer

Suppose $q_t \to q_*$ and, at every output token, the largest logit of $R(q_*)$ exceeds the runner-up by a strictly positive margin. Continuity then gives a neighborhood in which the argmax is constant. Eventually the answer stabilizes. If the limiting logits tie, even a convergent semantic trajectory can keep crossing the decision boundary. If the winning class is wrong, the stable answer is wrong.

More quantitatively, if each logit is $L_R$-Lipschitz and the smallest winning margin at $q_*$ is $\delta > 0$, every $q$ with $\|q - q_*\| < \delta/(2L_R)$ preserves all winning classes. These constants are not known certificates for a trained network unless actually bounded. The implication explains what must connect latent stability to task stability.

## 5.5 What is deliberately not guaranteed

Every stationary point is protected, including incorrect minima and saddles. At an exact wrong root, changing only $M$ or $w$ cannot escape. Noise, restarts or a workspace-dependent energy could move it, but would define a different method and require revised guarantees. An arbitrary additive workspace force would destroy the central invariant-fiber property.

A neural energy does not automatically have a known $\beta$, a PL region, a meaningful minimum, or a correct decoder. Floating-point computation also gives approximate rather than symbolic stationarity. These are material limits of the theorem, not reasons to reject the architecture before testing it.

# 6 How to build and train the proposed model

## 6.1 A first implementation that can answer the research question

Use a standard input encoder, a tokenwise semantic latent, a scalar energy network with cross-token attention, a logit head, and a bounded recurrent workspace. Aggregate energy contributions with a documented length normalization; changing sequence length must not silently rescale all gradients. Initialize $q_0$ from the input or a learned seed conditioned on the input, and initialize $w_0$ to a bounded state. Use the same initialization rule in matched baselines.

For an initial implementation, apply the bounded low-rank metric of equation (9) independently to token blocks. Feed $q_t, g_t, x, w_t$ to the workspace network and predict metric factors from $q_t, w_t, x$. Specify whether gradient features inside the workspace are differentiated or detached; either can be a training approximation, but neither should be concealed. Do not detach $g_t$ from the energy parameters in the semantic update if the energy is expected to learn through that update.

This is new model code, not a promised drop-in transformation of a pretrained TRM, Llama or Qwen checkpoint. Existing TRM data preparation, evaluation and reporting can be reused. Native TRM's detached recurrent carry is not automatically suitable for training this architecture.

## 6.2 Why learning an energy is more than naming a scalar

For training pair $(x, y)$, unroll a horizon $T$ and minimize, as a basic objective,

$$\mathcal{L}_{\text{traj}} = \sum_{t \in \mathcal{T}} \lambda_t \ \mathrm{CE}(R_\theta(q_t), y), \qquad \lambda_t \geq 0, \quad \sum_t \lambda_t = 1. \tag{12}$$

$\mathcal{T}$ is a prespecified collection of intermediate and final depths. Labels are used in training and offline evaluation, not as an inference-time correctness gate. Variable training horizons and multiple initial states can encourage improvements over depth, but are training design choices to be shared fairly with controls.

Backpropagating through $g_t = \nabla_q E_\theta$ requires mixed derivatives $\partial_\theta \partial_q E_\theta$ and Hessian-vector products. A scalar-energy forward plus a gradient calculation is substantially more expensive than one ordinary recurrent forward. Truncated backpropagation may be needed, but the cut locations and detach operations change optimization and must be documented. At inference the gradient with respect to $q$ is still required even though parameter gradients are not.

Cross-entropy on the trajectory does not force all minima of $E$ to be correct. A constant energy would produce no iterative motion; a sufficiently strong encoder/readout might still fit examples. Therefore measure improvement over the initial state, remove or reset workspace in controlled ablations, and compare equal-capacity encoder baselines. Persistent workspace that has no causal benefit is not evidence for the hypothesis.

## 6.3 Optional energy-shaping objectives and their limitations

One possible addition is a ranking loss

$$\mathcal{L}_{\text{rank}} = \max\{0, \tau + E(q^+; x) - E(q^-; x)\}, \qquad \tau > 0,$$

where $q^+$ and $q^-$ are explicitly constructed training representations of a correct and corrupted candidate. Another is a stationarity penalty $\|\nabla E(q^+; x)\|^2$. These are *options*, not an already settled recipe. They require a well-defined candidate-to-latent construction; a free learned "good state" encoder could collapse or make the ranking task irrelevant to states reached by inference. If used, the encoding must be shared with the trajectory representation and checked for accessibility, discrimination and nontrivial energy structure. Stationarity alone does not imply a minimum.

The first meaningful study should freeze a complete training recipe after a bounded development phase, rather than continually add losses until a favorable result appears. Strong energy-based reasoning baselines should receive comparable energy supervision and exploration opportunities. EBR's projected geometry and exploration training must be respected when reproducing that method; Euclidean gradient descent with our energy is an architecture-matched control, not automatically a faithful EBR reproduction [18, 19].

### 6.4 Step-size control: conditional theory or a real acceptance rule

A capped learned $\alpha$ is practical but not a formal descent certificate when $\beta$ is unknown. An alternative is Armijo backtracking. For $d = -Mg$, start from a proposed step and shrink until

$$E(q + \alpha d) \le E(q) - c\alpha g^\top M g, \qquad 0 < c < 1.$$

For nonzero $g$, smoothness and positive definiteness imply acceptance for sufficiently small steps. A finite retry cap may fail; specify that failure policy, for example retaining $q$ and recording the event. Every retry costs another energy evaluation. Differentiating through acceptance branches is a separate training issue; fixed-step training followed by backtracking inference is also a distribution change to measure, not a free theorem.

The core semantic update should remain Euclidean for the first specification. Adding normalization or projection changes the update and its proof: constrained stationarity and a Riemannian or projected-gradient analysis are then required. Similarly, a halting rule based on $\|g\| < \varepsilon$ detects near-stationarity, not correctness. Evaluate the accuracy/cost tradeoff, including false early stops.

# 7 Why this architecture might help, and how it could fail

The motivation is not that contraction is always harmful. It is that a useful history-dependent search process and a stable answer interface need not obey the same dynamics. The table states hypotheses that the project can actually test.

| Possible gain | Mechanism | Evidence needed and failure mode |
|---|---|---|
| Stable completed answers | Workspace influence is zero at a stationary semantic state and small in the one-step bound near it. | Measure correctness retention during extended rollout and workspace interventions. Wrong stationary answers are protected too; small gradient alone is insufficient. |
| Fewer iterations | History predicts useful directions or curvature scales through $M_t$. | Compare accuracy versus iterations with stateless metrics and stateful optimizers. Extra energy derivatives or workspace cost may erase a loop-count gain. |
| Better finite-budget accuracy | Different positive metrics follow different paths and may reach more useful basins before budget exhaustion. | Compare exact solve rate at matched total cost across seeds. The method cannot escape an exact wrong stationary point through $M$ alone. |
| Depth generalization | A common energy can limit destructive switching as computation continues. | Evaluate horizons longer than training and larger tasks. Unknown smoothness, numerical error, and energy/task mismatch can defeat the hoped-for benefit. |
| Persistent useful memory | Workspace retains information not recoverable from current $q$ alone. | Reset or shuffle workspace at selected depths, and compare an equally expressive stateless metric. A decorative oscillator or extra capacity is not the claimed mechanism. |
| Multiple internal representations | An invariant product fiber allows many full states with the same semantic answer. | Show that nonstationary workspace is useful during search. This does not establish multiple-solution sampling or a learned semantic manifold. |
| Practical real-valued modules | Energy, workspace and metric predictor can use familiar attention and MLP components. | Measure hardware runtime and memory. Second derivatives are costly, and pretrained compatibility is not automatic. |

## 7.1 A useful optimization analogy, with its limits

On a quadratic energy $E(q) = \frac{1}{2}q^\top Hq$ with $H \succ 0$, ordinary gradient descent has update matrix $I - \alpha H$. A perfect preconditioner $M = H^{-1}$ gives $q' = (1-\alpha)q$ and reaches the minimum in one step when $\alpha = 1$. This illustrates why geometry can matter more than the number of loops. Our bounded low-rank metric need not approximate $H^{-1}$ well, especially when curvature is ill-conditioned or indefinite. The example is motivation, not a performance prediction for a neural energy.

A workspace can estimate curvature from past gradients, which immediately links the proposal to quasi-Newton methods and learned optimizers. L-SR1 is particularly relevant because it learns positive-semidefinite preconditioning with secant information [21]. The contribution cannot be merely “remember history and precondition gradients.” It must establish a useful reasoning architecture, training strategy, or semantic stability/search tradeoff beyond these controls.

### 7.2 The strongest plausible achievement

A compelling outcome would be a model whose persistent workspace measurably improves solving at equal cost, while extended computation and workspace perturbations leave already stationary, correct semantic states stable. A weaker but still informative outcome is a clear failure: the metric adds cost without beating Adam or a stateless learned metric, or the learned energy develops wrong roots that dominate the error. The latter would motivate a different energy landscape or the richer attracting-set approach, not a claim that persistent motion itself is useful.

# 8 QRR, related work, and the other architectural proposals

## 8.1 What QRR contributes to the question

QRR [6] uses a prescribed gauge symmetry, with per-token phase transformations and a gauge-invariant readout. Its complex hidden and memory states transform together. An equivariant update maps equivalent states to equivalent states, so the quotient evolves without dependence on the arbitrary representative. This is a principled way to allow motion that does not alter the quotient answer.

For two complex arrays $h, m \in \mathbb{C}^{S\times d}$, the generic full real dimension is $4Sd$, not $2Sd$. A free $U(1)^S$ action has $S$ orbit directions, giving generic quotient dimension $4Sd - S$. At states with nontrivial stabilizers, orbit dimension changes; dimension counting requires that qualification. Removing $S$ chosen directions cannot guarantee that all slow or oscillatory modes disappear from the much larger quotient.

Crucially, a pure gauge phase cannot act as an independently useful causal clock for quotient computation when exact equivariance holds: different representatives of the same quotient state have the same future quotient trajectory. QRR may still be a useful inductive bias, but "free gauge oscillations carry otherwise missing reasoning signals" is not established by the symmetry. QRR also recognizes remaining quotient modes; this is not an undisclosed mathematical loophole.

Our proposal differs by allowing workspace to affect semantic trajectories *before* stationarity. Its decoupling is at roots, not an all-time quotient invariance. That gives a concrete role to the workspace but weakens the protection away from roots. Neither design dominates the other by theorem.

## 8.2 Audit of the supplied suggestions

| Suggestion or assertion | What survives careful analysis |
|---|---|
| Learned quotient coordinates | A valid candidate if a learned representation separates semantic and nuisance variables. If $q' = f(q)$ never depends on workspace, the latter cannot improve future semantic computation. An invertible change of coordinates or stop-gradient does not by itself impose the required forward independence. |
| Triangular Jacobian by stop-gradient | Stop-gradient changes the derivative used for training, not the actual forward response to an intervention. True triangular structure needs a forward architectural restriction. Its diagonal workspace block is not necessarily the identity. |
| Per-head $U(1)^{SH}$ | A legitimate richer symmetry. It removes the selected relative-head phase information as well as nuisance variation; it does not automatically absorb arbitrary quotient oscillations. Independent useful interactions must be represented in invariant features. |
| Non-Abelian $SO(k)$ or $U(k)$ | A broader architectural option using matrix-valued transport. Stabilizers, alignment ambiguities, representation capacity and polar-factor conditioning must be analyzed. Increasing group dimension is not a universal cure for the quotient spectrum. |
| Horizontal Jacobian regularization | Potentially useful after correctly defining tangent spaces and projections at both endpoints. Sampled trace estimates, finite power iterations and observed trajectory contraction do not certify a uniform operator-norm bound or global contraction. |

| Suggestion or assertion | What survives careful analysis |
|---|---|
| Cross-entropy implies monotone evidence | False in general. Cross-entropy does not force logits to improve at every recurrent step or force the quotient Jacobian to have real, contractive eigenvalues. |
| A singular scalar phase connection | Normalizing a small complex projection can be poorly conditioned; the exact behavior depends on epsilon handling and implementation. Treat phase slips as a failure hypothesis, not an observed QRR defect. |
| Complex layers preclude efficient attention | Too strong without implementation analysis. Phase canonicalization and real embeddings can sometimes reorganize computation. Rebuilding an equivariant model is a real cost; categorical FlashAttention impossibility is not established. |

### 8.3 Where novelty is already constrained

Energy minimization for iterative reasoning predates this proposal: IREM and EBR are direct references [19, 18]. Learned preconditioners and stateful optimizers are direct algorithmic controls [21]. TANGO already combines energy-driven and tangential computation in GNNs [20]; Appendix A explains what could still change for a recurrent reasoner. Projected neural differential equations likewise make tangent constraint preservation an existing design tool [22].

These comparisons have different review depths. QRR's full method and the EBR full-text method were inspected for the present direction. TANGO and L-SR1 were checked through their primary proceedings descriptions; a theorem-by-theorem and official-code audit remains necessary before claiming an implementation-level distinction. The earlier TRM/FPRM/InfiLoop and scheduling comparison is retained in Appendix K. No literature search proves exhaustive novelty.

### 8.4 What the critic accepted

The independent agent review accepted the stationary-fiber construction, bounded metric, conditional descent proof, and expanded attracting-set analysis after qualifications. It judged direct architectural implementation worthwhile. It explicitly did **not** accept a claim of a new optimization principle, guaranteed faster reasoning, or A* publication readiness. The review is internal AI criticism, not external peer review.

The surviving research question is precise: can a task-trained, persistent, causally useful workspace improve energy-based reasoning while its semantic coupling vanishes at stationarity, and does this offer a better accuracy/cost/retention tradeoff than strong stateful optimizers? If the answer is no, the mathematical invariant alone is not enough for a method paper.

# 9 Version 2.0 comparison design and living record

**BASELINE PLAN — Section 1 adds the current residual-space comparison and review findings.**

## 9.1 One architectural comparison, not a staircase of tiny gates

Implement the shared energy/readout/initialization backbone and compare the following search mechanisms under a frozen training recipe and data split:

1. gradient descent on the shared energy, plus a faithful published energy-reasoner reference where feasible;
2. momentum/Adam and a strong learned stateful optimizer or secant-based preconditioner;
3. a learned positive metric depending on current $q, x$ but no persistent workspace;
4. the proposed persistent-workspace positive metric;
5. QRR and TRM as external architectural comparisons, with their full cost and training differences stated.

These controls separate the benefit of energy learning, generic optimization history, extra parameters, and the specific workspace interface. For mechanisms that need extra parameters, use both an identical-energy controlled comparison and an approximately parameter/cost-matched end-to-end comparison. Equal architecture widths alone do not ensure equal compute.

The primary outcome is exact solve rate versus *total* inference cost and wall time. Report energy forwards, gradient evaluations, workspace calls, backtracking retries, peak memory and training cost. A method using fewer nominal loops but more time is not a demonstrated efficiency improvement. Include several training seeds and uncertainty over independent puzzles, not over correlated recurrent steps.

Use a reasoning task with meaningful headroom and a size/depth generalization setting. Sudoku is useful for continuity and exact verification but should not be the sole near-ceiling headline. Final task mix, model sizes and accelerator budget remain to be fixed before launching training; this document has not silently committed paid compute or invented a completed protocol. Extended rollouts, workspace-reset interventions and gradient/margin traces belong inside this same substantial study, not as months of prerequisites.

## 9.2 Decisive interpretations

If the persistent workspace beats stateless metrics but not Adam or a learned optimizer, the result supports history-dependent optimization, not our specific architectural novelty. If it wins only at more compute, report that tradeoff. If removing workspace has no effect, its internal motion is not useful evidence. If it improves finite-budget solving and retains correct stationary outputs across seeds and harder cases at matched cost, investigate which information in its history causes the gain and what the method adds beyond existing optimization.

The old E4/E5 clock protocols remain reproducible optional studies. They can explain the original TRM dynamics, but their success or failure does not gate this new model. Do not reinterpret the completed pilot as evidence that a learned energy or solution manifold already exists.

## 9.3 How future updates should be recorded

For every new result preserve: the prespecified question; code/checkpoint/data hashes; seed and split; full cost definition; exact executed configuration; raw output location; independent report verification; uncertainty and denominators; surviving alternative explanations; and the resulting decision. Separate changes motivated by evidence from changes motivated by a research preference. This version's pivot is a deliberate architectural choice informed by new literature and the user's priorities, not a new empirical discovery.

| Version | Date | Change |
|---|---|---|
| 1.0 | 6 Nov 2026 | Integrated pilot, TRM feedback example and optional clock protocol; no new trained model. |
| 2.0 | 12 Nov 2026 | Reorganized around persistent-workspace energy reasoning; added mathematical primer, proofs, training/cost design, QRR critique and expanded attracting-answer-set appendix. Preserved old evidence and protocols as history. No new trained-model result. |
| 2.1 | 12 Nov 2026 | Four critique rounds, supplied QRR review assessment, residual-space architecture and exact separation, local attraction, SNR/random-mixing novelty limits, and an operator prototype. No new trained-model result. |

# A The attracting-answer-set architecture, developed fully

**PROPOSED ALTERNATIVE — not discarded; no trained implementation or result**

## A.1 Start with the distinction that motivated the project

Imagine solving a puzzle and then continuing to think about it. You might recheck constraints, reorganize a representation, or traverse equivalent internal descriptions without changing your answer. A fixed-point model describes completion by one unchanged hidden vector. An attracting-answer-set model describes completion by entering a region or surface on which the computation can continue safely.

Let the full hidden state be $s \in \mathbb{R}^n$. For fixed input $x$ and a target answer $y$, define the correctness region

$$\mathcal{C}_{x,y} = \{s : D(s) = y\}.$$

For tasks with multiple acceptable outputs, replace equality with a task validity condition. A desired attracting answer set $\mathcal{M}_x$ should satisfy three logically separate properties:

1. **Correctness:** $\mathcal{M}_x \subseteq \mathcal{C}_{x,y}$, or all its decoded outputs satisfy the task's validity condition.
2. **Invariance:** starting in $\mathcal{M}_x$ keeps the trajectory in $\mathcal{M}_x$.
3. **Attraction:** for an appropriate basin of initial states, $\operatorname{dist}(s_t, \mathcal{M}_x) \to 0$.

Correctness is a relation between geometry and the task. Invariance is a property of the update rule. Attraction concerns nearby trajectories. None follows automatically from the others. A stable wrong cycle can satisfy the last two. A collection of correct states need not be invariant. An invariant set can repel nearby states.

There is one further discrete-output qualification. Attraction to a correct set does not by itself imply eventual correct decoding if the set lies on a decision boundary. A robust correctness neighborhood, for example a uniform positive logit margin on a compact target set, supplies the missing implication.

## A.2 A manifold of equilibria is different from a moving manifold

If every $s \in \mathcal{M}$ satisfies $F(s) = s$, then $\mathcal{M}$ is a manifold of equilibria. Different initial states can settle to different points, but motion stops once a point is reached. Alternatively, $F$ may move states along $\mathcal{M}$. A periodic orbit is an example; a higher-dimensional invariant surface need not be periodic. The second possibility expresses the original intuition most directly.

For continuous time, write $\dot{s} = f(s)$, where $\dot{s}$ is the instantaneous velocity. At a smooth point of $\mathcal{M}$, its tangent space $T_s\mathcal{M}$ contains velocities compatible with remaining on the manifold to first order. Its normal space contains directions leaving it. We would like normal errors to shrink while tangent computation remains available. This is often called transverse or partial stability; a full normal-hyperbolicity theorem would additionally require precise rate domination and regularity conditions, which are not assumed here.

## A.3 The simplest complete example: approach a circle and keep rotating

Take $s = (u, v) \in \mathbb{R}^2$, radius $r = \sqrt{u^2 + v^2}$, and define

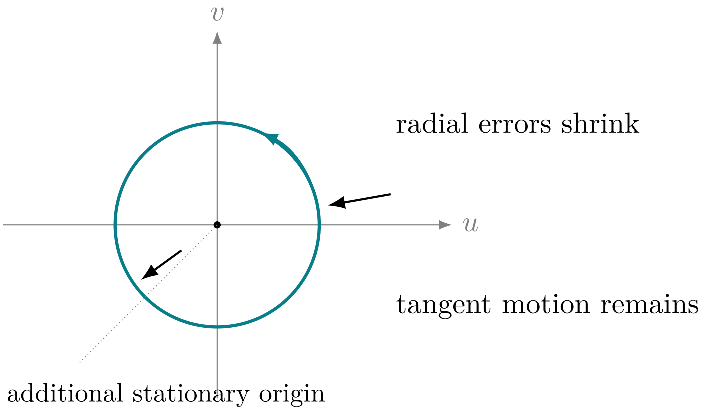
$$V(s) = \tfrac{1}{4}(r^2 - 1)^2, \qquad J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad \dot{s} = -\kappa \nabla V(s) + \omega J s,$$

with $\kappa > 0$ and $\omega \neq 0$. The potential $V$ is zero exactly on the unit circle. Its gradient is $(r^2 - 1)s$, a radial vector. Multiplication by $J$ rotates a vector by a quarter-turn, so $Js$ is tangent to the circle and orthogonal to $s$.

In polar coordinates the dynamics become

$$\dot{r} = -\kappa(r^2 - 1)r, \qquad \dot{\vartheta} = \omega \quad (r > 0).$$

If $0 < r < 1$, then $\dot{r} > 0$; if $r > 1$, then $\dot{r} < 0$. Thus every nonzero initial radius approaches one, while the angle keeps changing. The unit circle attracts nearby states and carries permanent motion. The origin is also stationary and stays there: even this simple example contains an unwanted critical state outside the desired attracting set.



The potential decreases according to

$$\dot{V} = \nabla V^\top \dot{s} = -\kappa \left\|\nabla V\right\|^2 = -\kappa(r^2 - 1)^2 r^2 \leq 0,$$

since $s^\top Js = 0$. Now compare two readouts. If $R(s) = \|s\|$, then $R \to 1$ and is exactly constant on the circle. If $R(s) = u$, then $R = \cos(\omega t + \vartheta_0)$ on the circle and keeps oscillating. The same stable hidden dynamics therefore permit either a stable or unstable answer interface.

**What to carry forward.** An attracting manifold protects a decoded answer only when the readout is aligned with that manifold. The geometry and the task representation must be learned or designed together.

## A.4 Energy descent plus tangent computation

The general continuous-time form is

$$\dot{s} = -K_\theta(s, x)\nabla V_\theta(s; x) + T_\theta(s, x), \tag{13}$$

where $K$ is symmetric positive semidefinite and

$$\nabla V(s; x)^\top T(s, x) = 0 \tag{14}$$

throughout a neighborhood, not merely at the desired solution. The first term reduces potential; the second moves along its level sets wherever the gradient is nonzero. Then

$$\dot{V} = -\nabla V^\top K \nabla V \leq 0.$$

If $V \geq 0$ and the flow is well posed, this inequality already implies forward invariance of the zero set $\mathcal{M} = \{s : V(s) = 0\}$: a trajectory starting at zero can neither increase nor become negative. No independent tangency assumption is logically necessary for that theorem.

However, the orthogonality condition checked *only at a critical point* is vacuous, since $\nabla V = 0$. It does not specify a valid tangent field there. A smooth construction that satisfies (14) throughout the neighborhood must be compatible with the zero-set geometry. Also, being tangent to a positive-energy level set is not the same as being tangent to a correct-answer set.

A tempting parameterization is $T = A(s,x)\nabla V$ with $A^\top = -A$. It gives orthogonality automatically because $g^\top A g = 0$. But it also gives $T = 0$ wherever $\nabla V = 0$, so it does not create persistent motion on a zero-energy manifold. It can be useful away from stationarity, but it misses the original goal if used alone. The circle field $T = \omega J s$ illustrates the distinction: it remains nonzero on the target circle although the energy gradient is zero.

### A.5 What LaSalle's invariance principle actually provides

Assume sufficient regularity for unique trajectories and a compact positively invariant region containing the trajectory. LaSalle's invariance principle says that trajectories approach the largest invariant subset of the zero-dissipation set

$$\mathcal{Z} = \{s : \nabla V(s)^\top K(s) \nabla V(s) = 0\}.$$

If $K \succ 0$, then $\mathcal{Z} = \{s : \nabla V(s) = 0\}$. If $K$ is only semidefinite, zero dissipation can occur with a nonzero gradient in the nullspace of $K$. Neither version says that the limiting set is the desired zero set or that it decodes correctly. The circle's stationary origin is a concrete reminder.

To obtain a task-relevant theorem one must exclude undesirable invariant subsets in the basin under consideration, establish attraction to the intended set, and align that set with the decoder. A proof that a scalar never increases is only one ingredient. Global guarantees for combinatorial reasoning should not be inferred from a local smooth construction.

### A.6 An implicit learned manifold: constraints instead of a prescribed group

A more flexible architecture learns a residual map

$$c_\theta(s;x) \in \mathbb{R}^r, \qquad \mathcal{M}_x = \{s : c_\theta(s;x) = 0\}, \qquad V(s;x) = \tfrac{1}{2}\left\|c_\theta(s;x)\right\|^2.$$

Think of $c$ as a vector of latent consistency errors. It need not be a known symbolic Sudoku constraint, but if it is learned, its connection to task correctness must be enforced by training and checked empirically. Let $J_c(s) = D_s c(s) \in \mathbb{R}^{r\times n}$. If $r < n$ and $J_c$ has full row rank near a zero, the implicit function theorem says that the zero set is locally a smooth manifold of dimension $n - r$.

A tangent velocity $v$ must satisfy $J_c v = 0$, because $\frac{d}{dt}c(s) = J_c \dot{s}$. The orthogonal projector onto the tangent nullspace is

$$P(s) = I - J_c(s)^\top \left(J_c(s) J_c(s)^\top\right)^{-1} J_c(s). \tag{15}$$

For any proposed neural velocity $v_\theta(s,x)$, define

$$\dot{s} = -\kappa J_c^\top c + P(s) v_\theta(s,x), \qquad \kappa > 0. \tag{16}$$

The first term is $-\kappa\nabla V$. The second removes components that would change $c$ to first order. Since $J_c P = 0$,

$$\dot{c} = -\kappa J_c J_c^\top c, \qquad \dot{V} = -\kappa c^\top J_c J_c^\top c.$$

If the smallest singular value of $J_c$ is uniformly at least $\sigma > 0$ throughout a forward-invariant region, then

$$V(t) \le e^{-2\kappa\sigma^2 t} V(0), \qquad \|c(s(t))\| \le e^{-\kappa\sigma^2 t}\|c(s(0))\|.$$

This is an explicit transverse residual-contraction result. Translating residual size to distance from $\mathcal{M}$ requires a local regularity/error bound. Full rank locally supplies such geometry near a regular point; a uniform global distance certificate needs more. Correctness still requires $\mathcal{M}_x$ and its neighborhood to have the intended readout.

This architecture can describe input-dependent curved solution sets without choosing $U(1)$ or any other group in advance. That flexibility is its attraction, and also its difficulty: the model must learn a meaningful manifold, not just an easy residual.

## A.7 Why a learned constraint can collapse, and what training must address

If the only loss rewards $c(s) = 0$, the identically zero function solves it. Then the “constraint” defines the whole state space, there is no normal contraction, and the projector formula’s rank assumption fails. Even a nonconstant residual can define a manifold of wrong answers. A separate decoder can also ignore the intended geometry.

A serious implementation therefore needs task supervision of trajectories, contrast between correct and corrupted candidate representations, and explicit evaluation of the rank and meaning of the residual. Rank encouragement or conditioning penalties can help avoid trivial geometry but do not ensure correctness. If symbolic task constraints are available, they can ground $c$; they may still have multiple valid solutions, continuous relaxations or spurious stationary points. Do not insert the unknown target answer into inference-time constraints.

To keep logits constant under tangent computation, a stronger local condition is

$$D_s R(s)\, T(s, x) = 0.$$

One may project a proposed velocity into the intersection of $\ker J_c$ and $\ker D_s R$, using their stacked Jacobian. This is expensive, may leave no useful tangent directions, and can become ill-conditioned. If the derivative of $R$ annihilates *every* tangent direction on a connected manifold, then $R$ is constant on that component. Annihilating only the chosen velocity gives constancy along those trajectories, a weaker claim. For discrete answers, preserving a margin neighborhood can be sufficient without keeping every logit exactly constant.

There is an important expressiveness tradeoff: globally forbidding tangent motion from affecting any future semantic output can turn it into irrelevant computation. Before completion, the normal update may still alter the answer and tangent history may influence future descent through state-dependent geometry. The design must demonstrate that this interaction helps solve, rather than merely conserving a label after it has already been computed elsewhere.

## A.8 Projection cost, singularities and finite time steps

Equation (15) assumes full row rank. A pseudoinverse gives $P = I - J_c^\dagger J_c$, a projector onto the Jacobian nullspace, but its smoothness requires constant rank. At a singular point the linear nullspace can misrepresent the nonlinear zero set. For example, if $c(u, v) = u^2 + v^2$, then $J_c(0, 0) = 0$ and its nullspace is all of $\mathbb{R}^2$, even though the zero set is only the origin.

Replacing $(J_c J_c^\top)^{-1}$ by $(J_c J_c^\top + \varepsilon I)^{-1}$ improves conditioning but generally gives $J_c P \neq 0$. That is approximate tangency, with a leakage term that must be bounded or measured. Jacobian-vector products, vector-Jacobian products and an $r \times r$ solve can avoid materializing every large matrix, but computation and higher-order training derivatives remain substantial. This is not a free architectural layer.

Most neural reasoners use discrete steps, not exact ODE solutions. A tangent Euler step $s' = s + \eta T(s)$ generally leaves a curved manifold by $O(\eta^2)$. On the unit circle, $T = \omega J s$ gives

$$\|s + \eta\omega J s\|^2 = 1 + \eta^2\omega^2,$$

so the radius grows even when the starting point is exactly on the desired set. Continuous-time invariance cannot simply be quoted as a discrete-network guarantee.

For the circle, the exact rotation $s' = \exp(\eta\omega J)s$ preserves radius; a split method can combine a controlled radial step with this rotation. For a learned manifold, one can use retraction or projection back onto $c = 0$, a constraint-preserving integrator, or a discrete acceptance/correction rule. Each adds assumptions, possible solver failures and cost. Projection onto $c = 0$ also remains useless for task correctness if the learned constraint is wrong.

### A.9 Why the GNN precedent does not end this direction

TANGO [20] already motivates energy-guided dynamics with a tangential component for graph learning. Therefore we must not claim that combining energy dissipation and tangent motion is a new mathematical principle. Projected neural differential equations also provide constraint-tangent constructions [22]. The fact that an existing method is used in GNNs does not prove it works as a recurrent reasoning architecture, nor does it make that transfer scientifically uninteresting.

The reasoning setting introduces specific requirements: the readout is a discrete candidate solution; computation continues for variable depth; a correct answer should remain correct; trajectories must still revise wrong hypotheses; and every extra gradient/projection must be charged to inference cost. These are different design pressures from simply preserving a learned graph representation. An input-conditioned constraint manifold aligned with a solution decoder, together with a discrete update that preserves it while retaining useful search dynamics, could constitute an architectural contribution if it solves these problems in a way existing methods do not.

The publication claim must rest on that mechanism and evidence. Merely applying the same energy-plus-tangent layer to Sudoku and renaming it would be weak. A useful study would adapt the strongest relevant GNN construction to the same backbone, compare it with the product-fiber architecture and energy/optimizer baselines, and isolate the contribution of readout alignment, discrete preservation and tangent workspace. Detailed TANGO theorem/code comparison remains pending; the current source review establishes overlap, not an exhaustive equivalence result.

### A.10 Product fibers versus learned attracting manifolds

| Question | Main proposal | Attracting-set alternative |
|---|---|---|
| Target geometry | $\{q_*\} \times \mathcal{W}$ at stationary semantic states. | Learned curved $\{s : c(s; x) = 0\}$, potentially without a global coordinate split. |
| Useful motion | Workspace changes preconditioning away from roots. | Tangent computation can move on consistency level sets and interact with normal search. |
| Protection | Exact semantic invariance at a root, by multiplication with zero gradient. | Requires correct tangent construction, readout alignment and discrete preservation. |
| Attraction | Conditional common-energy descent; stronger convergence assumptions needed. | Residual contraction under uniform rank and regional invariance; task alignment still needed. |
| Main risk | Wrong roots are frozen; optimizer novelty may be limited. | Residual collapse, rank loss, projection cost, readout leakage and numerical drift. |
| Research value | A simpler direct test of useful history plus a stable semantic interface. | A more general answer to the original manifold question, deserving a separate architectural branch. |

**What to carry forward.** The attracting-answer-set idea remains a serious research option. Its strongest version learns meaningful input-dependent geometry and preserves a task-aligned set in the actual discrete model. The GNN precedent raises the standard for the contribution; it does not rule out a new reasoning architecture.

# B The actual machine: what TRM updates

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

## One Sudoku, two arrays, one shared network

For one Sudoku puzzle, TRM stores two arrays of real-valued features. We call them:

- $h$: the **answer state**, called `z_H` in the code. A small output head reads this array to predict the board.
- $w$: the **working state**, called `z_L`. It influences future updates of $h$, but is not read directly by the output head.

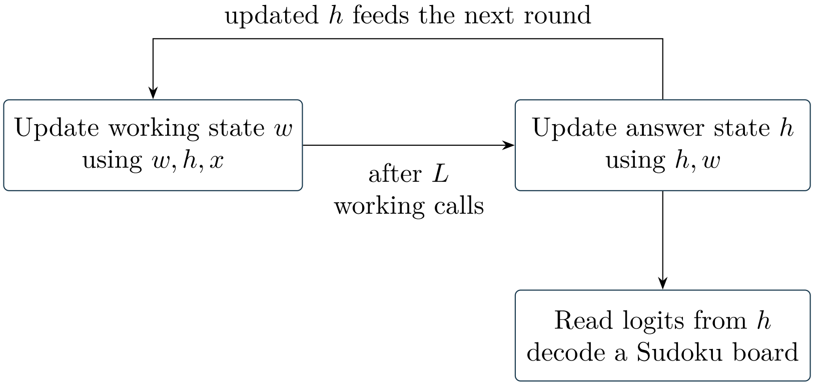
These names describe their computational roles. They do not mean that $h$ literally contains Sudoku digits or that $w$ contains interpretable written reasoning. In the current configurations, each state for one puzzle has shape $97 \times 512$: 81 cell positions plus 16 prefix positions, each with 512 features. A batch of eight has shape $8 \times 97 \times 512$.

Let $x$ denote the fixed input embedding of the puzzle, including the model's positional/prefix information. Let $G$ denote the shared two-block neural module. Ignoring the fixed positional arguments in the notation, the source implements:

$$w \leftarrow G(w + h + x) \qquad \text{working update,} \tag{17}$$

$$h \leftarrow G(h + w) \qquad \text{answer update.} \tag{18}$$

**It is the same $G$ in both equations.** Its parameters are shared. The arguments differ. The actual implementation adds the current state and its injection *before* the shared block stack; this matters when constructing a faithful mathematical example.


updated $h$ feeds the next round
Update working state $w$ using $w, h, x$
after $L$ working calls
Update answer state $h$ using $h, w$
Read logits from $h$ decode a Sudoku board

The output head computes scores (*logits*) for each possible cell token from $h$. Taking the largest score at each cell gives the predicted board. Changing $w$ alone therefore leaves the *immediate* logits exactly unchanged. It can still change later logits, because the next answer update uses $w$. That simple fact motivated the first interventions.

**Code anchor:** models/recursive_reasoning/trm.py, especially the reasoning module's additive injection and the inner model's `forward`. This description is grounded in that code, alongside TRM [1].

## The word "step" was hiding three different quantities

The existing pilot logs use *step* for a native macro-step. The newer theoretical note uses one *outer update*. They are not interchangeable.

| Unit | What happens at the native setting | Block calls |
|---|---|---|
| One working update | Apply $G$ once to update $w$; hold $h$ fixed. | 2 |
| One outer update | Update $w$ six times, then update $h$ once. | 14 |
| One native macro-step | Perform three outer updates; emit the normal model output. | 42 |
| Normal 16-macro budget | 48 outer updates in total. | 672 |
| Long pilot: 64 macros | 192 outer updates in total. | 2,688 |

Here $L = 6$ is the number of working updates and $H = 3$ is the number of outer updates per macro. Each $G$ call contains two block layers. Thus

$$\underbrace{(L+1)\times 2}_{\text{one outer}} = 14, \qquad \underbrace{H(L+1)\times 2}_{\text{one macro}} = 42.$$

Unless explicitly stated otherwise, every depth in the *completed pilot* sections is a macro count. The future study uses outer counts and block budgets explicitly.

## Why we preserve both states and bypass resets

The full continued computation depends on $(h, w)$, the input and the update phase. Copying only $h$ does not produce an identical future computation. Our branches copy both arrays at a specified boundary and keep the puzzle fixed.

The standard ACT wrapper can halt an example and reset its state for subsequent processing. That is appropriate for ordinary operation, but would interrupt a long-trajectory experiment. Our adapter continues the same internal state without those resets. It preserves the underlying recurrence. It does not train new parameters or silently replace pretrained weights with random ones.

## What training does, and what this project does

The upstream model is supervised with target answers at macro outputs and uses a learned halting-related $Q$ head. Within a macro, early outer iterations run without gradients; gradients pass through the final outer iteration, and carried states are detached before the next macro. This is a training-memory choice, not a proof of convergence. The initial-state vectors are checkpoint-stored buffers.

Our current experiments freeze the pretrained weights. Ground-truth boards are used *afterward* to measure correctness, not to choose a branch, perturbation direction or schedule. A Sudoku rule checker tests validity using the puzzle's constraints; that is distinct from inspecting the reference solution.

**What to carry forward.** More working updates means more calls using an unchanged answer state. Reading intermediate outputs only observes the computation; changing the number of working updates changes the computation itself.

# C Fixed points, stable answers and moving states

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

## A fixed point need not be unique

Write $s = (h, w)$ for the full internal state, and $F$ for a complete update rule. A fixed point satisfies $F(s_*) = s_*$. If the model reaches that exact state, the next update leaves it unchanged. The equation may have no solutions, one solution, or many solutions. “Fixed point” by itself does *not* mean “one unique representation for every answer.”

An **attracting** fixed point also draws nearby states toward it. Its **basin** is the collection of starting states that eventually approach it. These are different questions: a useful state can exist but be difficult to reach; an incorrect state can be attracting.

## The output is a coarse view of the state

Let $D(h)$ be the board decoded from $h$. The states producing one board $y$ form the set

$$\mathcal{S}_y = \{(h, w) : D(h) = y\}.$$

This set can be large because $w$ is not directly read, and many different $h$ vectors have the same largest logit at each cell. The set need not be a smooth *manifold*. A manifold is a set that locally looks like a lower-dimensional continuous space; proving that property requires more than observing unchanged predictions.

Staying in $\mathcal{S}_y$ means retaining the same decoded board. It does not mean $s$ has stopped. It also does not mean $y$ is correct. A collection of hidden representations of one solution is distinct from a puzzle having several valid solutions.

## A small example of the original intuition

Consider a purely illustrative continuous-time system, with state $(r, \theta)$:

$$\dot{r} = -(r - 1), \qquad \dot{\theta} = 1 \pmod{2\pi}.$$

The radius approaches 1 while the angle keeps rotating. The attracting object is a circle, not a stationary point. A readout that depends only on the limiting radius could settle while the full state keeps moving. This explains why attraction to a set is a coherent idea; it is not a model we have trained or evidence that TRM uses a circular representation.

In dynamical-systems language, **transverse stability** concerns disturbances away from such an orbit or set; motion along it can remain. **Output contraction** asks for readout differences to shrink, a weaker requirement than shrinking every state difference. **Partial contraction** concerns selected relations or components. These concepts motivate what to measure, but transferring their guarantees requires checking their assumptions [14, 15, 16].

**What to carry forward.** The original point-versus-set question remains meaningful. Our experiments have not identified a manifold or proved any contraction property. The November 6 direction focused on the coupled update rule because it could be directly manipulated. The current architecture direction is in Section 1.

# D The experiment ledger: what actually exists

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

These IDs are stable references for this notebook. They are not new runs. All completed trained-model experiments used the same third-party MLP checkpoint, with the eight-puzzle cohorts nested inside the 32-puzzle cohort.

| ID | Status | Question / setup | Result or next decision |
|---|---|---|---|
| E0 | Checked | Adapter, zero controls, manifests, strict checkpoint loading, synthetic smoke tests. | Instrumentation evidence; synthetic accuracies are not research results. |
| E1a | Observed | 32 puzzles; native BF16; 16 macros. | 25/32 correct at the endpoint. No correct-to-wrong transition at logged macros. |
| E1b | Observed | Same 32; FP32; continue to 64 macros. | 25/32 correct at 16, 27/32 at 64. No logged loss after solving. |
| E2 | Observed control | Eight puzzles; fork at macros 4, 8, 16; zero perturbation; 16-macro branches. | All 1,152 branch records exactly matched the natural continuation. |
| E3 | Observed | Same eight; perturb only $w$ by 1% relative norm; random and gradient directions. | Immediate logits unchanged. Some wrong trajectories changed; no correct-at-fork answer was lost. |
| E4 | Implemented; not run | 1,024 new dev puzzles; two checkpoints, two precisions; equal-budget $L = 6$ versus $L = 12$. | The next result planned on November 6; now optional. See Appendix I. |
| T0/T1 | Proved / checked | Original transient example (T0), then shared-core feedback example (T1). | Possibility examples, not tests of trained networks. |
| E5 | Proposed only | If justified by E4: equal numbers of working and answer updates, differently interleaved. | No real-model launcher or outcomes for this follow-up yet. |

## Why this table matters

There are four completed main pilot runs, not dozens of independent datasets. Repeating several branches on the same puzzle is useful for paired comparisons, but does not multiply the number of independent puzzles. The eight examples are exploratory and unusually small for a research claim.

The current suite has **39 passing software/mathematical tests**, recorded at the research snapshot. They check things such as correct state copying, report completeness, budget accounting and finite-difference agreement. They are not 39 successful research experiments. Strict loading of the attention weights and agreement between native and densely observed recurrence establish implementation compatibility, not Sudoku performance.

No FPRM or InfiLoop backend has been implemented in this repository. No learned stabilizer, new training loss or adaptive scheduling controller has been trained or evaluated.

## E What the natural-continuation pilot tells us

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

**OBSERVED: E1a and E1b. Depths on this page are native macro-steps.**

| Run | Correct at 16 | Correct at end | Ever-correct then lost |
|---|---|---|---|
| BF16, stop at 16 | 25/32 | 25/32 | No observed losses |
| FP32, continue to 64 | 25/32 | 27/32 | 0/27 puzzles |

### Follow the same puzzles, rather than just the percentages

At FP32 macro 16, 25 puzzles were correct and seven were wrong. By macro 64:

- all 25 initially correct puzzles were still correct and had stayed correct at every logged macro;
- two of the seven initially wrong puzzles had become correct;
- five were still wrong at the endpoint.

Accuracy therefore rose from 78.125% to 84.375%, an improvement of 6.25 percentage points. The reported paired puzzle-bootstrap 95% interval was $[0, 15.625]$ percentage points. This is a small exploratory sample, not a precise population estimate.

### "No observed loss" has a specific scope

Outputs were logged after each native macro, not after every answer update inside it. A board could have become wrong between two logged outputs and recovered before the next one. E1b therefore establishes no observed loss on its *macro observation grid*. It does not establish permanent correctness or stability at every internal update.

The reports list 0/334 and 0/1,608 correct-to-wrong transitions for BF16 and FP32 respectively. Those many transitions reuse the same puzzles; they are correlated. They should not be described as hundreds or thousands of independent examples. For continuation from the usual budget, the useful count is 0/25 FP32 puzzles losing correctness through macro 64.

### Precision changes more than the final aggregate score

Both runs achieved 25/32 at macro 16, but not on exactly the same puzzles: one was correct only in BF16 and another only in FP32. Full predicted boards matched on 24/32 puzzles. BF16 uses lower-precision arithmetic than FP32; equal aggregate accuracy does not imply identical trajectories.

We have **not** measured BF16 continuation beyond macro 16 in this pilot. It would be incorrect to use the FP32 extension as evidence about every numerical regime. The run times also overlapped, so their recorded wall times are not a controlled speed comparison.

**What to carry forward.** The pilot gave us a recovery phenomenon, not the retention failure we initially expected to repair. That is why the project did not proceed directly to training a stabilizer.

# F What the perturbation experiment changed

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

**OBSERVED: E2 and E3. A sensitivity experiment, not an inference method.**

### The intervention, step by step

At each predetermined macro (4, 8 or 16), we saved the same $(h, w)$ and made several copies. One copy continued unchanged. In the others we replaced $w$ with $w + \delta$, keeping $h$ untouched. The nonzero intervention was chosen so that

$$\frac{\|\delta\|}{\|w\|} \simeq 0.01.$$

In plain terms, the change had about 1% of the working state's norm. Since the current output depends only on $h$, all immediate logits had to remain identical. That equality was explicitly checked, rather than assumed from an approximate similarity score.

**Random + and −** use opposite directions along a random perturbation. **Gradient + and −** use opposite directions derived from a short future-margin objective. The margin compares the score of the model's current predicted token with alternatives; the probe uses current predictions, not the reference answer. Gradient plus follows the direction that locally reduces that future margin; minus reverses it. A two-macro lookahead and backward pass cost extra computation. Neither sign guarantees better or worse correctness.

All branches then continued for 16 macros, beyond the two-macro gradient probe. We compared each branch with the matching clean continuation at the same offset. We did not search for a favorable fork after looking at labels.

### First, the zero control

E2 set $\delta = 0$. All 1,152 branch records matched the corresponding natural trajectory, including predictions and logged observables. This validates copying and continuation. It is not evidence that perturbations help.

### Then, the nonzero result

At every fork, five of the eight puzzles were correct and three were wrong. All 96 nonclean puzzle/fork/arm interventions met the norm tolerance and preserved immediate logits exactly.

| Fork | Arm(s), each separately | Final solve | Loss from correct | Recovery from wrong |
|---|---|---|---|---|
| 4 | Random ±, gradient ± | 6/8 | 0/5 | 1/3 |
| 8 | Gradient + | 5/8 | 0/5 | 0/3 |
| 8 | Random ±, gradient − | 6/8 | 0/5 | 1/3 |
| 16 | Random ±, gradient ± | 6/8 | 0/5 | 1/3 |

The clean final solve count was 6/8. Grouped arms above share the same numerical result; they are not pooled samples. No correct-at-fork puzzle changed its board relative to clean at any observed branch offset.

### One actual puzzle makes the distinction concrete

One puzzle, with ID beginning `e524867c`, was wrong at all three forks. Its clean trajectory first became correct at macro 17. Under the gradient-plus intervention:

| Continuation | What happened | Meaning |
|---|---|---|
| No intervention | First correct at macro 17. | The usual 16-macro budget had stopped just before recovery. |
| Perturb at macro 4 | First correct at macro 7. | Recovery was accelerated in this example. |
| Perturb at macro 8 | Still wrong at branch endpoint, macro 24. | Recovery was prevented or delayed beyond the measured window. |
| Perturb at macro 16 | First correct at macro 17. | The measured solve time was unchanged. |

This is **not** a case where a correct solution was destroyed. At macro 8 the answer was already wrong. The intervention changed whether it became correct later. Nor does failure through macro 24 prove that it would never recover.

The final eight-puzzle gradient-plus difference at fork 8 was $-12.5$ percentage points, with a reported pointwise 95% bootstrap interval of $[-37.5, 0]$. With one changed outcome and only eight examples, this does not establish a reliable stage-dependent law. Other perturbation directions sometimes accelerated the same puzzle too. The case was selected for inspection after the results were known.

### The narrow conclusion that is supported

Changing information not read by the current output can change later answers. That is evidence that *present readout equivalence does not imply future dynamical equivalence.* A direction invisible to the current output is not necessarily irrelevant to the computation.

This experiment did not establish a useful learned controller, spontaneous failure mechanism, stable solution manifold, or successful retention repair. It also did not show that gradient directions are systematically better than random directions.

### Why not simply do more of the same?

The desired practical motivation was to preserve already correct answers. Neither natural continuation nor these perturbations exposed that failure in the tested cohort. Stronger attacks could manufacture failures without explaining normal behavior. Close prior work also already studies late completion and directional stability [4]. The next useful experiment should address the unresolved protocol and dynamics questions at a substantial scale.

**What to carry forward.** "An intervention changed future correctness" is the observed fact. "We identified and repaired the cause of overthinking" would be an unsupported interpretation.

# G Why the early diagnostic programme changed direction

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

### The apparent disagreement with InfiLoop

InfiLoop v2 reports a TRM retention problem: its Figure 1 shows 19.7% solved-puzzle loss after 36 additional outer updates. The cohort contains puzzles both compared models solve within the initial 36 updates. A separate 20.9% statistic concerns cells, not whole solved puzzles [3].

Our pilot did not replicate that setup:

| Dimension | Our longer pilot | InfiLoop diagnostic |
|---|---|---|
| Model | Third-party MLP TRM | Self-attention TRM |
| Observation | One readout per three outer updates | Outer-update readouts |
| Cost per outer | 14 block layers | 26 block layers |
| Precision | FP32 continuation | Exact figure precision not verified |
| Cohort | 32 predetermined dev puzzles | Jointly solved cohort of two models |

We inspected the released code and relevant figure text, but did not identify the exact Figure-1 TRM checkpoint and complete reproduction script. The pilot neither reproduces nor refutes that figure. Its 64 macros already cost 2,688 block evaluations, so "we simply did too little total computation" is not an established explanation.

### Observation frequency and update schedule are different interventions

Suppose a native trajectory emits correct boards at macros 16 and 17. Looking between those outputs might reveal an incorrect intermediate board. We have learned something by observing more often, but we have not changed the model and have not shown failure of its usual output policy.

By contrast, changing $L$ from 6 to 12 inserts six extra working updates before each answer refresh. It changes the path of the computation. Matching the other paper's 26-block cost this way is *not* an exact reproduction of its checkpoint or training schedule.

### The literature also narrowed the available claim

*When Recursive Models Finish Computing* studies extended TRM recurrence, directional stability and perturbations, including the same MLP checkpoint. FPRM Appendix F already investigates inner/outer compute allocation. TAPS already adapts update strength [4, 2, 5]. A small perturbation study or a schedule accuracy table would be insufficient on its own.

**What to carry forward.** We chose a controlled schedule comparison to distinguish observation effects, numerical/checkpoint effects and continuation-schedule effects. A positive result would still need a predictive mechanism and a useful intervention to support a new method.

# H Historical coupled-feedback insight

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

**HYPOTHESIS ABOUT TRM; SUPPORTED BY A MATHEMATICAL POSSIBILITY EXAMPLE**

### The intuition: local improvement can worsen the feedback

During working updates, $w$ repeatedly uses the same held-fixed $h$. Then $h$ changes using the resulting $w$. More working updates may make $w$ better adjusted to the *old* $h$, while making the next change of $h$ larger or badly coordinated with the coupled system.

Think of $h$ as a current proposal and $w$ as working notes tied to that proposal. Refining the notes for longer need not help if the next proposal update changes what those notes should be responding to. This is an analogy for the equations, not a claim that the features are interpretable notes.

The hypothesis is therefore: **some limits of continued reasoning may depend on the timing and strength of feedback between the two states, even when the same exact fixed states remain available.** We have not yet shown this mechanism in a trained network.

### Existence and attraction are separate: a one-number example

Both rules below have the fixed point $s_* = 0$:

$$s_{t+1} = 0.5 s_t, \qquad s_{t+1} = 1.2 s_t.$$

Starting at 0.1, the first gives $0.1, 0.05, 0.025, \ldots$; the second gives $0.1, 0.12, 0.144, \ldots$. The fixed point has not disappeared. Nearby states simply move differently. Starting at *exactly* zero stays at zero under either rule.

The current TRM question has this same logical structure, with a high-dimensional coupled state. We ask whether changing the update schedule can preserve a state that solves the fixed-point equation while changing whether nearby trajectories approach it.

### What we can prove for TRM's update structure

Call one working-only update $A$ and one answer-only update $B$. One outer update with $L$ working calls is

$$F_L = B \circ A^L.$$

Here $A^L$ means apply $A$ repeatedly $L$ times, then apply $B$. Since $A$ leaves $h$ unchanged and $B$ leaves $w$ unchanged, any exact fixed point of $F_6$ is also an exact fixed point of $F_{12}$. Appendix L gives the short proof.

This result says *nothing* about correctness, attraction, the basin, or behavior from ordinary finite-budget states. It also concerns one outer update; a state fixed after a three-outer macro could instead be part of a three-cycle.

## A concrete coupled example: same solution, different stability

We constructed an artificial scalar system with TRM's shared additive wiring:

$$G(v) = v(v-1)(0.7v+0.2), \qquad x = 1,$$

$$w \leftarrow G(w+h+1) \quad L \text{ times}, \qquad h \leftarrow G(h+w).$$

It uses one number for each state instead of a large array. At $(h, w) = (0, 0)$, both updates return zero. Each subsystem is locally stable when the other state is held fixed. Nevertheless:

| Schedule | Local growth factor | What happens near the origin |
|---|---|---|
| $L = 6$ | 0.670489 | Nearby errors eventually shrink. |
| $L = 12$ | 1.254233 | An error direction grows. |
| $L = 12$, damp answer update to 80% | 0.694608 | Local attraction is restored. |

The displayed factor is the *spectral radius*: the largest magnitude of the linearized update's eigenvalues. Below one means local asymptotic attraction here; it does not mean every single step decreases every error norm. Appendix L.5 derives the values and damping threshold.

Damping means taking only part of the proposed answer-state change:

$$h_{\text{new}} = h + \eta(h_{\text{proposal}} - h).$$

For example, $\eta = 0.8$ accepts 80% of the proposed change. Damping is a standard idea and an essential baseline, not the claimed novelty.

## What the critic accepted, and what it did not

The critic required the example to use *one shared function with additive inputs*, matching the actual code's wiring. An earlier, more general construction did not meet that constraint and was rejected. The corrected example and fixed-set argument survived review. They provide a reason to test a coupled-feedback explanation.

They do not construct actual post-normalized Transformer weights, demonstrate a trained-model instability, identify ODE stiffness, or show that a latent instability changes a discrete Sudoku answer. A scalar existence example is not a performance result.

**A hard limit:** timing alone cannot move an exact state fixed by both $A$ and $B$. If that state decodes incorrectly, repeating or damping those unchanged updates still leaves it there. A constant wrong board, however, is not proof of an exact latent fixed point.

**What to carry forward.** The idea retained on November 6 was a causal account of coupled feedback, with a useful intervention predicted in advance. The mathematical example establishes possibility. For this historical diagnostic branch, E4 was intended to establish whether there is a relevant trained-model effect to pursue. It does not gate the current architecture programme.

# I Optional historical E4 protocol, preserved in full

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

**E4: OPTIONAL; IMPLEMENTED; FULL TRAINED-CHECKPOINT STUDY NOT YET RUN**

### The question

From the same state reached by normal TRM computation, does changing the balance of working updates and answer updates alter retention or recovery, when both continuations get the same block budget?

### For each puzzle, do exactly this

1. Run the native model for 16 macros: 48 outer updates, or 672 block evaluations. This prefix is shared by the two branches.
2. Save *both* states $(h, w)$ at that fixed boundary. The fork time is predetermined, not the first time a reference answer happens to be correct.
3. Make two identical copies. Continue one with $L = 6$ and the other with $L = 12$. Keep weights and input fixed; add no perturbation and perform no training.
4. Give *each* continuation 2,184 block evaluations. Log a board after every answer update, and retain separate records of native macro boundaries.
5. After the run, use labels to measure what happened. Correct-at-fork puzzles measure retention; wrong-at-fork puzzles measure recovery.

| Continuation quantity | $L = 6$ | $L = 12$ |
|---|---|---|
| Block calls per outer update | 14 | 26 |
| Outer / answer updates in branch | 156 | 84 |
| Working updates in branch | 936 | 1,008 |
| Total branch block calls | 2,184 | 2,184 |
| Common-grid spacing | 13 outer updates | 7 outer updates |

This is an equal-cost *schedule* comparison. It does not isolate just one variable such as the number of working calls: the number of answer refreshes also changes. Within a checkpoint, each shared block has the same shape. Equal block counts are not a claim of equal FLOPs between the MLP and attention architectures, nor equal wall-clock time. Readout and embedding costs are counted separately.

The common grid is every 182 blocks. Intermediate grid points can lie between native macro outputs. At the final budget, both branches have completed a multiple of three outer updates, so their endpoint phases align.

### Which puzzles and models?

Use the same 1,024 project-development puzzles in all four conditions, beginning at offset 32 to exclude the exploratory prefix. The split seed is zero. Conditions are the MLP and attention checkpoints, each in BF16 and FP32. Attention/BF16 is the prespecified primary condition.

Each numerical condition computes its own native prefix. The two branches *within* that condition share the exact state. Cross-precision comparisons do not start from identical latents. With one checkpoint per architecture, architecture and training/checkpoint differences cannot be separated.

### Three kinds of “losing a solution”

1. **Native output loss:** a correct board at a native macro is followed by a wrong board at a later native macro. This concerns the model’s usual output policy.
2. **Intermediate excursion:** after a correct native output, a dense intermediate readout becomes wrong. If every later macro in the window stays correct, ordinary macro logging would miss that excursion.
3. **Dense first-solve loss:** after the first correct readout at any outer update, a later dense readout becomes wrong. This can include a fleeting intermediate solution never emitted as a native macro output.

These are not interchangeable denominators or claims. The fixed-fork schedule comparison uses the same correct-at-fork cohort in both arms, before either treatment has an outcome. We never select just the puzzles an arm later solves.

There is also a descriptive native observation audit: follow the first eligible native solve for 36 further native outer updates, requiring a complete window. That costs 504 blocks. It is not the 36-times-26-block window of the InfiLoop figure. Late or never-solving puzzles excluded from this audit must be reported.

### What is measured and how it will decide the next step

The primary effect is $L = 12$ minus $L = 6$ in the fraction of correct-at-fork puzzles that become wrong at *any dense observation.* The prespecified practically meaningful harm is two percentage points. Appendix M explains the paired uncertainty calculation.

Secondary outcomes include final correctness, recovery from wrong-at-fork states, native macro-output loss, and the fraction of the block budget during which the current answer is incorrect. The last statistic holds the most recent $h$ readout constant while the next computation runs; the terminal output has zero duration. It is a computation-budget statistic, not a wall-clock user experience measure.

The sample size is fixed. At least 300 correct-at-fork puzzles is an adequacy flag, not permission to keep collecting until a favorable answer appears. The full four-condition design costs 20,643,840 sample-block evaluations, excluding output heads, input embeddings and analysis.

# J Historical E4/E5 decision rules

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

| Possible E4 outcome | Consequence |
|---|---|
| The primary 95% lower bound is above +2 percentage points. | Support a practically meaningful harmful schedule effect in this setting. Replicate before assigning a mechanism. |
| The primary upper bound is below +2 percentage points. | Exclude that prespecified harmful effect size under this protocol. Do not keep the claim alive by strengthening attacks. |
| The interval includes +2 percentage points. | Inconclusive about the target effect size, even if the estimate differs from zero. |
| Intermediate errors occur, but native macro outputs retain correctness. | An observation-frequency finding. This alone does not motivate repairing native output retention. |
| $L = 12$ damages answers while native $L = 6$ stays stable. | Altered-schedule fragility. It is not evidence that native TRM already has this defect. |
| Both schedules retain answers with informative bounds. | Stop this retention-repair direction for these conditions. |
| A schedule improves final correctness or recovery at equal cost. | Consider a useful allocation method; require a label-free policy and strong baselines. |

### The conditional E5 follow-up: same amounts, different order

If E4 provides a reason, compare repeated pairs $(6, 6)$, $(3, 9)$ and $(9, 3)$ of working-update counts. In all cases, a pair contains 12 working calls and two answer calls. Repeating 78 pairs gives each arm 936 working updates, 156 answer updates and 2,184 block calls. This controls the number of answer refreshes that differs in E4.

The scalar example predicts a better eventual linearized convergence rate for uneven allocation, but also warns that short-term amplification can be worse. Reversing $(3, 9)$ to $(9, 3)$ changes the starting phase; it does not change the eigenvalues of the linearized period map. A difference between those arms would need a phase/transient/nonlinear explanation. E5 is not yet part of the trained-model launcher.

### What would make this a serious research contribution?

A mechanism should predict which states benefit, or predict a component-specific damping rescue, *before* seeing their continuation. It must outperform simple fixed schedules, extra native continuation, ordinary damping, TAPS and verifier stopping where applicable, with all diagnostic and search compute charged. It then needs new puzzles and checkpoints beyond those used to design it.

If the only effect is damage from schedules the model never trained on, we have not produced a useful method. A toy and a schedule sweep do not establish conference-level novelty. Stopping an unsupported line is part of the plan.

# K Earlier literature review: November 6 snapshot

**HISTORICAL RECORD — November 6 snapshot; current architecture priority is Section 1.**

These are orientation notes, not interchangeable claims of full reproduction. The project reviewed the closest work to narrow the question; it has implemented only a TRM experimental backend.

## The three starting architectures

**TRM** [1] repeatedly uses a small shared network to update two latent states. It supplies our actual code and frozen checkpoints. It does not enforce a globally contractive fixed-point equation. Our question concerns what happens when that learned recurrence continues or its schedule changes.

**Fixed-Point Reasoners / FPRM** [2] uses architectural choices including residual scaling and a convergence-based halting mechanism to support deep looping. Its theoretical sufficient conditions should not be confused with unconditional guarantees about every learned checkpoint. Appendix F already compares inner/outer allocation at approximately fixed compute. Thus "more inner computation can be worse" is not a new general observation.

**InfiLoop** [3] retains a weighted running summary of past recurrent states rather than relying only on the latest proposal. Its memory mechanism motivates examining retention, but it introduces persistent aggregation state that must be included in any faithful dynamical description. Its Figure-1 protocol differs materially from ours, as Section G explains. We have not reproduced its baseline figure.

## The work that most directly limits our novelty

**When Recursive Models Finish Computing** [4] examines extended recurrence and directional stability on 1,000 Sudoku puzzles, including our MLP checkpoint. It separates unfinished computation from persistent failure. We cannot claim generic late recovery, perturbation sensitivity or directional stability as our new contribution. The project's full-text review was central to choosing a more specific next question.

**TAPS** [5] adapts update scaling using trajectory information. Its update definition and inner/outer control and refresh choices were inspected; the nested recurrence remains. A new timing/feedback controller would need to compare against it. Its code still requires inspection before implementing a fair comparator.

**QRR** [6] studies convergence after identifying a symmetry in the latent state: hidden motion can continue while the relevant quotient representation settles. **Invariant-Measure Reasoners / ImR** [7] instead reads a prediction through an expectation under a stable distribution. These occupy versions of the original "stable answer without a stationary hidden state" idea. ImR's primary abstract was reviewed; a detailed implementation comparison remains open.

**What to carry forward.** The possible contribution is not "loops can move without changing the answer" or "allocation matters." It would be a predictive explanation of coupled feedback in trained reasoners, tied to a useful intervention at matched cost.

## Other comparators and dynamical-systems tools

**Readout Feedback / RoFB** [8] introduces feedback from intermediate readout probabilities into recurrent dynamics. **Input-Conditioned Search / ICS** [9] changes the input conditions used by the update map. Both matter if we move toward steering wrong trajectories. The project has checked RoFB's introduction/method overview and ICS's primary abstract; their detailed methods and theoretical assumptions still need review.

**Learning to Compute Is Not Learning to Retain or Reuse** [10] separates these abilities through state interventions. Retention failure and causal interventions are therefore not new categories introduced by this project. Random depth/position supervision is a relevant training baseline if training is later justified.

**Transient amplification** [11, 17] explains how a system that eventually contracts can first magnify disturbances. Matrices whose directions interact can have this behavior; eigenvalues alone do not measure every short-term change. The distinction matters because we care about finite-budget answers, not only asymptotic convergence.

**Coupled PDE solvers** [12, 13] offer a useful analogy: solving one component more accurately while holding information from another component fixed does not automatically improve the joint iteration. The mathematical precedent is grounding for our hypothesis, not a new numerical-analysis theorem or evidence about TRM weights.

**Partial, transverse and output contraction** [14, 15, 16] formalize different quantities that might stabilize without requiring every coordinate to stop. They are potential theoretical tools for a later result. No metric or bound satisfying those theories has been established for our checkpoints.

## Directions being kept in reserve

A learned schedule, high-state damping, a readout-preserving stabilization loss, attracting-set training or a state-displacing escape mechanism are possible later branches. None is the immediate experiment. Exact common fixed states cannot be escaped by timing alone; adding a new feedback signal or changing the map would be a different intervention and would require the relevant comparators.

The broader resource index also covers DEQs, path independence, normalization/gain control, stochastic stopping, probabilistic TRM, attractor models and torus networks. Those are reading leads rather than results of this project. The maintained full index is docs/RESOURCES.md; it records review depth and source links so incomplete paper review is not mistaken for established novelty clearance.

# L Historical TRM proofs and coupled-feedback example

This appendix preserves the earlier diagnostic theory, not the guarantees of the newly proposed architecture. The current main argument does not require reading this appendix. Its purpose is to make the theoretical claims explicit, checkable and reusable. All maps below hold the input fixed.

## L.1 The earlier theory: why stable eigenvalues were not enough

Before the coupled-feedback example, the project used this two-coordinate linear illustration (T0):

$$s_{t+1} = Js_t, \qquad J = \begin{pmatrix} 0.8 & 5 \\ 0 & 0.9 \end{pmatrix}, \qquad s_0 = (0, -0.1), \qquad m_t = 1 + (s_t)_1.$$

Here $m_t$ is an artificial decision margin, declared positive for a correct decision. Both eigenvalues, 0.8 and 0.9, are below one in magnitude. The state eventually approaches zero and the margin approaches one. Nevertheless, the second coordinate pushes the first strongly enough that the margin temporarily becomes negative before recovering. This is transient amplification: eventual convergence does not guarantee retention at every finite time. It is not a Sudoku measurement or a TRM weight construction.

## L.2 The original selective-stability idea, now conditional

Rather than shrinking every direction, the initial ambition was to control changes that threaten future decisions. For a smooth future margin $m_k$, a perturbation $\delta$ in an allowed linear subspace $S$, and $\|\delta\| \le \varepsilon$, Taylor's theorem gives

$$m_k(s+\delta) \ge m_k(s) - \varepsilon \, \|P_S \nabla m_k(s)\| - \tfrac{1}{2} C_k \varepsilon^2.$$

$P_S$ keeps the gradient components in the allowed perturbation directions (here, the working-state coordinates). $C_k$ bounds curvature along the relevant line segments. To protect a whole board for several future times, a positive bound is needed for every cell, competitor token and time. For rigor the competitor is fixed; taking a maximum over tied competitors introduces nonsmoothness.

This standard inequality suggests what sensitivity to measure, but its curvature constants have not been verified for our model. It is not a certificate. It preserves a selected answer, which could already be wrong. Using future positive baseline margins also assumes future survival; that cannot be advertised as predicting a spontaneous failure in advance.

The pilot and close prior work did not justify turning this idea directly into a new loss. It remains a conditional direction, with damping, generic robustness, depth supervision and existing stability regularizers as necessary baselines. T0 and this bound explain the project's original motivation; T1 is the newer, narrower feedback example below.

## L.3 Why doubling the working count preserves an outer fixed point

Let $s = (h, w)$. The map $A$ changes only $w$ and $B$ changes only $h$. Define $F_L = B \circ A^L$. If $F_L(s) = s$, the working coordinate after $A^L$ must already equal its initial value, because the subsequent $B$ cannot change it. The answer coordinate was unchanged throughout $A^L$. Therefore $A^L(s) = s$, and subsequently $B(s) = s$.

Conversely, if both $A^L(s) = s$ and $B(s) = s$, their composition fixes $s$. Hence

$$\mathrm{Fix}(F_L) = \mathrm{Fix}(A^L) \cap \mathrm{Fix}(B).$$

Repeating a map that returns $s$ does not move $s$, so for positive integer $m$,

$$\mathrm{Fix}(F_L) \subseteq \mathrm{Fix}(F_{mL}).$$

This proves the 6 → 12 statement. Equality for arbitrary positive $L$ needs an additional condition: for example, that the held-$h$ working dynamics have no nontrivial periodic points. Contraction would suffice for that premise; it is not assumed for trained TRM.

An exact fixed point of the native macro $F_6^3$ may be a three-cycle of $F_6$. Likewise, $\mathrm{Fix}(A^6)$ can contain periodic states that are not fixed by one $A$. These distinctions prevent overextending the claim to every schedule.

If $A(s) = B(s) = s$, any composition of these maps leaves $s$ unchanged. Residual damping also leaves zero proposals unchanged. The argument applies to a deterministic finite-precision implementation with identical operators as well: roundoff is not an automatic escape mechanism from an *exact implemented* fixed state.

## L.4 What a Jacobian tells us

Near an equilibrium, a smooth update can be approximated by

$$F(s_* + e) - s_* = Je + O(\|e\|^2),$$

where $e$ is a small displacement and $J$ is the matrix of first derivatives, or Jacobian. Its spectral radius $\rho(J)$ is the largest eigenvalue magnitude. If $\rho(J) < 1$, the equilibrium is locally asymptotically stable for this smooth discrete system. An eigenvalue magnitude above one gives an unstable direction. At equality, the linear test alone is insufficient.

The largest singular value $\sigma_{\max}(J)$ instead measures the largest Euclidean norm gain in *one* linearized update. It can exceed one even when $\rho(J) < 1$. Thus eventual stability and immediate shrinking are distinct.

## L.5 The shared-core example and its stability threshold

For $G(v) = v(v-1)(0.7v+0.2)$, we have $G(0) = G(1) = 0$, $G'(1) = 0.9$ and $G'(0) = -0.2$. The origin is a common equilibrium of the individual updates. With $h$ fixed, a small displacement in the working state obeys

$$\delta w_{j+1} = 0.9\,\delta w_j + 0.9\,\delta h.$$

After $L$ calls, write $q = 0.9^L$ and $u = \sum_{j=1}^{L} 0.9^j = 9(1-q)$. Then

$$\delta w_{\text{new}} = q\,\delta w + u\,\delta h, \quad \delta h_{\text{new}} = -0.2(\delta h + \delta w_{\text{new}}).$$

In $(h, w)$ order the matrix is

$$J_L = \begin{pmatrix} -0.2(1+u) & -0.2q \\ u & q \end{pmatrix}, \quad T = \operatorname{tr} J_L = -2 + 2.8q, \quad D = \det J_L = -0.2q.$$

For a real quadratic $\lambda^2 - T\lambda + D$, both roots are strictly inside the unit circle exactly when

$$1 - T + D > 0, \qquad 1 + T + D > 0, \qquad 1 - D > 0.$$

Here these reduce to $3 - 3q > 0$, $-1 + 2.6q > 0$ and $1 + 0.2q > 0$. Since $0 < q < 1$, stability is equivalent to

$$q > \frac{5}{13}.$$

Thus $L = 1, \ldots, 9$ are stable and $L \geq 10$ are unstable. At $L = 6$, $\rho(J_6) = 0.670489$; at $L = 12$, $\rho(J_{12}) = 1.254233$. The strict inequalities also show that sufficiently small parameter changes preserve this distinction. It is not dependent on one fragile equality.

## L.6 A predicted damping rescue

Replace the answer update by $h_{\text{new}} = h + \eta(G(h + w_{\text{new}}) - h)$, where $0 < \eta \leq 1$. The top row of $J_L$ becomes

$$(1 - \eta(1.2 + 0.2u),\ -0.2\eta q).$$

Its trace and determinant are $T_\eta = 1 - 3\eta + (1 + 1.8\eta)q$ and $D_\eta = q(1 - 1.2\eta)$. The nontrivial stability inequality is

$$\eta < \frac{2(1+q)}{3 - 0.6q}.$$

For $L = 12$ this is $\eta < 0.906137$; choosing 0.8 yields $\rho = 0.694608$. This is a mathematical rescue prediction for the example, not a tested intervention on pretrained TRM.

## L.7 Equal counts can still have different dynamics

Two successive outer updates with working counts $(a, b)$ have derivative $J_b J_a$ at the common equilibrium. Each row below uses 12 working and two answer calls. If we price a core call as two TRM blocks, each period has 28 block-equivalent calls. This is a schedule-accounting analogy, not measured FLOPs for the scalar function.

| Counts $(a, b)$ | $\rho(J_b J_a)$ | $\sigma_{\max}(J_b J_a)$ |
|---|---|---|
| $(6, 6)$ | 0.449555 | 2.258716 |
| $(3, 9)$ | 0.106288 | 2.982494 |
| $(9, 3)$ | 0.106288 | 0.852055 |

The uneven allocation improves the asymptotic local linearized rate per period in this example. It does not guarantee smaller error at every finite time or for every initial direction. In particular, $(3, 9)$ permits greater first-period norm amplification than $(6, 6)$.

The matrices $J_9 J_3$ and $J_3 J_9$ have the same eigenvalues. Reversing the order is a phase shift of the same periodic policy. Differences from an identical starting state can concern transients, phase and nonlinear basins; they cannot establish different local asymptotic stability merely from reversal.

## L.8 The corresponding high-dimensional calculation

At a *common equilibrium of the individual additive updates*, let $P$ be the derivative of $G$ at the working evaluation and $R$ its derivative at the answer evaluation. They need not coincide because the same network is evaluated at different inputs. Define $U_L = \sum_{j=1}^{L} P^j$. Repeating the perturbation calculation gives

$$J_L = \begin{pmatrix} R(I + U_L) & RP^L \\ U_L & P^L \end{pmatrix}.$$

If $P^L \to 0$, then $I + U_L \to (I - P)^{-1}$, and the effective answer feedback tends to $R(I - P)^{-1}$. Separate stability of $P$ and $R$ does not guarantee stability of this coupled quantity. This is a standard coupled-iteration mechanism, not a new general theorem.

Away from the equilibrium, derivatives change along the trajectory and require ordered products. One local eigenvalue is not a complete explanation of finite-horizon correctness. A trained-model diagnostic would need to predict actual future outcomes beyond simpler features such as depth, logit margins, $Q$, residuals and precision.

## L.9 A limited one-step bound

With $h$ fixed, let $a = A_h^6$ act on the working state, $w_6 = a(w)$ and $w_{12} = a(w_6)$. If $a$ is $K$-Lipschitz on the relevant region, then $\|w_{12} - w_6\| \le K \|w_6 - w\|$. If the answer update is $M$-Lipschitz in $w$ there, its difference is at most $MK \|w_6 - w\|$. The constants are unmeasured and may be large. This is not a long-horizon retention certificate or a correctness test.

# M What the uncertainty numbers mean

For the primary E4 comparison, take the $n$ puzzles correct before the fork. For each puzzle, record two binary indicators: did it ever become wrong under $L = 6$, and did it ever become wrong under $L = 12$? A paired comparison asks about the *same* puzzles, not two unrelated samples.

Let $b$ count puzzles lost only under $L = 12$, and $c$ count puzzles lost only under $L = 6$. Puzzles lost in both arms, or in neither, cancel in the difference. The estimated excess loss is

$$\widehat{\Delta} = \frac{b - c}{n}.$$

For illustration only, $b = 15, c = 5, n = 500$ would give two percentage points. Those are not project results.

The implementation bounds the two discordant-event probabilities conservatively with simultaneous exact binomial (Clopper–Pearson) bounds and subtracts them. The resulting interval can remain nonzero in width when no discordant event was observed. This avoids interpreting a zero-width small-sample bootstrap interval as proof of no possible population effect. Check clock.py for the implemented construction.

## Why the threshold is two percentage points

The research decision concerns a prespecified practically meaningful size, not simply whether the estimate is nonzero. If a 95% interval is $[0.2, 1.5]$ percentage points, it excludes a two-point harmful effect under the protocol even though it is above zero. If it is $[1, 4]$, it is inconclusive about the two-point threshold. These are explanatory examples, not measured results.

Zero losses among 300 independent puzzles yields a one-sided 95% upper bound

$$1 - 0.05^{1/300} \simeq 0.00994.$$

This is the source of the "at least 300 solved puzzles" adequacy flag. It is not a full power analysis for the paired schedule difference and does not change the fixed 1,024-puzzle sample size.

## What is and is not independent

The resampling/counting unit is a puzzle, not a Sudoku cell, time point, branch or repeated observation. Exact input deduplication removes identical rows but not all symmetry-related dependence. Public-test reuse and checkpoint selection also limit generalization. There is only one checkpoint per architecture, so the analysis does not estimate training-seed variability.

Continuous incorrect-budget differences use a paired puzzle bootstrap with 2,000 resamples. Intervals are pointwise. The three secondary checkpoint/precision conditions and other outcomes are exploratory, not simultaneous family-wise significance claims. The earlier pilot used bootstrap intervals; its tiny eight-puzzle zero differences do not prove equivalence.

# N Data, weights and exact provenance

## Repository history and evidence boundaries

| Source | Revision / role |
|---|---|
| Upstream TRM | `c01103738605ba39d1430519b1ee0c62f4c707f8` |
| Committed pilot reports | `98689fb58da8bcb2791820f5674bd13d77ffc61b` |
| E4 implementation | `2ccf2814a70a929cd3f91b6073513fda3b4c7842` |
| Theory/code snapshot | `89425187566a5c649661407c7905ef4f70b0329b` |

The original model, training and dataset code are preserved. Added experiments live under `experiments/decision_stability/`. The current notebook is a later documentation commit; its research snapshot above identifies the evidence it explains, avoiding a self-referential document hash.

## MLP checkpoint used in E1–E3 and planned E4

gaoxin492/TinyRecursiveModels-Sudoku-Extreme-mlp, revision

```
256f32fcbe7123e8bf8c449410773a5ad311dbc5
step_16275 SHA256:
20e9dc7ebf83b9b41a8b3f58f5fd94ee3a7eb0b0d245bdeeb14e2f1488d1daaf
all_config.yaml SHA256:
811e64a1fa2e923e601791339b736bb3bab8bd350778ba116908d7965c021777
```

This is a third-party reproduction, not an official author checkpoint. Published source agrees with the pinned TRM source and loading is strict. Training dependencies, selection history and whether the specific weight file is EMA are not independently established. The publisher's reported score is not reproduced by our 32-puzzle sample.

## Attention checkpoint for E4

gaoxin492/TinyRecursiveModels-Sudoku-Extreme-att, revision

```
33991b882368f425916d625a9ee614a7a85efe76
step_21700 SHA256:
932b5e0df4e90021c21a064d4e716154086c46fda7afa676ea294484a30e38c3
all_config.yaml SHA256:
9e595c896d6484e9dbb3c3963d11bac224dd91d70eb040f11de04ac8303be6b3
published trm.py SHA256:
3454818b64a1abb45c062782d380aa4bb560e2cb35db9cc309cf367ca3ac262c
```

This is also a third-party reproduction, not a verified InfiLoop Figure-1 baseline. Strict loading and native/dense recurrence agreement were checked in BF16 and FP32 on synthetic inputs. Those checks do not measure benchmark accuracy. Downloaded Python source is compared, never executed.

## Sudoku data and population

Source: sapientinc/sudoku-extreme, public `test.csv` only.

```
Revision: 58942f96baeb572ca3127e2a9e9c70f330783d6b
CSV SHA256:
a2fd52aea23d331d5b4ee723c856236e838a9fb9a70e66f4e0e0cf26c338c6a8
```

All 422,786 rows were prepared before exact deduplication and the project split. There were 422,786 exact unique inputs; development contains 211,393 puzzles. Split seed zero and a deterministic ID order select the first 32 for E1, nested first eight for E2/E3, and offset 32 through the next 1,024 for E4.

The project has not inspected its held-out project-test outcomes. However, splitting an already public test set does not make a pristine external test: upstream evaluation may already have used it. There has been no training-overlap or Sudoku-symmetry contamination audit.

Inputs are 81 cell tokens: digits use tokens 2–10, blank is 1 and padding is 0. No training download or augmentation was used for the pilot. Prepared provenance and input hashes are embedded in production manifests.

## Pilot settings and retained identifiers

All four pilot runs used batch size four and two CPU threads. E1a used BF16; E1b/E2/E3 used FP32. E2/E3 forked at macros 4, 8 and 16, then continued 16 macros. E3's gradient probe was two macros; relative perturbation norm was 0.01 with 5% tolerance. Smaller puzzle sets were nested and noise seed was zero.

The worked example's full ID is

```
e524867ce8ff8ad0f9697194a08d65e3d935a8a3ff432cba98eac9c83585364b
```

Run names are `pilot-native-32`, `pilot-fp32-32-depth64`, `pilot-zero-8` and `pilot-targeted-8`. The committed log records their manifests as complete and independently rechecked. Raw outputs, weights and data are Git-ignored and should be retained separately by the person executing the study. Do not reconstruct missing raw measurements from this summary.

Recorded wall times were 461.1, 542.2, 372.8 and 520.6 seconds, respectively. Some runs overlapped. These times describe execution history, not a fair hardware or precision speed benchmark.

# O Historical diagnostic commands and code map

**OPTIONAL EXISTING CODE — these commands do not implement the new architecture.**

Run commands from the repository root. The experiment environment used Python 3.12 and PyTorch 2.7.0; CPU quickstart instructions are in the README. The full E4 launcher normally requires a CUDA-compatible PyTorch installation and a GPU. The CPU-only quickstart does not supply GPU support.

## Reproduce the mathematical example

```
.venv/bin/python -m experiments.decision_stability.toy
.venv/bin/python -m experiments.decision_stability.coupled_toy
```

The first command reproduces T0's transient margin; the second prints T1's coupled-feedback quantities. Neither loads a trained checkpoint. T1's tests use finite differences through the nonlinear recurrence to check the derived Jacobian.

## Run the frozen study

Reuse the pinned MLP checkpoint and prepared evaluation data described in PILOT.md; these are prerequisites, not bundled assets. Then:

```
.venv/bin/python scripts/fetch_attention_checkpoint.py
TRM_DEVICE=cuda TRM_BATCH_SIZE=8 TRM_OUT=runs/clock-study-v1 \
  bash scripts/run_clock_study.sh
```

The output root must be new. The launcher checks pinned checkpoint/config hashes, strict loading and device availability; runs attention BF16 first, then the three secondary conditions; verifies reports; and checks identical IDs across completed conditions. It does not download benchmark data, train weights, automatically resume, or expand the sample based on outcomes.

Path/runtime overrides include `TRM_PYTHON`, `TRM_DATA`, `TRM_MLP_CONFIG`, `TRM_MLP_CHECKPOINT`, `TRM_ATT_CONFIG`, `TRM_ATT_CHECKPOINT` and `TRM_THREADS`. Preserve scientific settings when relocating the run. An explicit CPU mode exists, but this is a substantial study, not a laptop micro-pilot.

## One condition, for transparent inspection

```
.venv/bin/python -m experiments.decision_stability.clock \
  --config checkpoints/gaoxin492-sudoku-att-33991b8/all_config.yaml \
  --checkpoint checkpoints/gaoxin492-sudoku-att-33991b8/step_21700 \
  --data data/sudoku-extreme-public/prepared \
  --device cuda --dtype bfloat16 --attention-backend math \
  --limit 1024 --offset 32 --partition dev --split-seed 0 --seed 0 \
  --batch-size 8 --low-cycles 6 12 --fork-outer 48 --branch-blocks 2184 \
  --followup-outer 36 --out runs/clock-att-bf16
.venv/bin/python -m experiments.decision_stability.clock_report \
  runs/clock-att-bf16
```

The run writes `manifest.json`, `prefix.jsonl`, `branches.jsonl`, `anchors.json`, `summary.json` and `report.md`. The independent report checks completeness and hashes and recomputes outcomes without loading a model. Keep raw files for audit; commit concise results and artifact hashes after execution.

## Where to look in the repository

| Path | Responsibility |
|---|---|
| `models/recursive_reasoning/trm.py` | Original state updates, readout, gradient cuts and ACT wrapper. |
| `models/layers.py` | Shared block operations, normalization and numerical choices. |
| `pretrain.py` | Original training, checkpoint and EMA handling. |
| `experiments/decision_stability/model.py` | Fixed-phase full-state adapter and gradient-enabled probe path. |
| `experiments/decision_stability/run.py` | Natural continuation and intervention pilot. |
| `experiments/decision_stability/report.py` | Pilot tables and independently recomputed reports. |
| `experiments/decision_stability/clock.py` | E4 schedules, dense observation, cohorts, costs and outcomes. |
| `experiments/decision_stability/clock_report.py` | Independent E4 reporting and validation. |
| `experiments/decision_stability/coupled_toy.py` | Corrected scalar coupled example. |
| `scripts/run_clock_study.sh` | Fixed four-condition E4 launcher. |
| `tests/decision_stability/` | Scientific/software invariants and integration checks. |

## Build or edit this document

The editable source is a single self-contained file:

```
docs/notebook/project_notebook.tex
```

Upload it to Overleaf and select **XeLaTeX**, or use a normal TeX Live installation with the packages declared in the preamble. There is no separate bibliography file or image asset to locate. Locally, from the repository root:

```
bash scripts/build_notebook.sh
```

The build produces `docs/notebook/project_notebook.pdf`. Check unresolved references and overfull boxes in the log, render the PDF and inspect its pages. Update the date, evidence snapshot and change log when scientific content changes; do not change E4's protocol silently.

**Verification scope.** The code snapshot reports 39 passing tests and successful repository checks. For changes limited to this guide, verify source links, commands, numerical consistency, successful LaTeX compilation and the rendered pages. Re-running a trained benchmark is not necessary to validate typography and must not be implied by a document build.

# P Glossary and notation

Symbols are local to each model. In particular, the new workspace $w$ is not the existing TRM tensor $z_L$, and the historical toy's scalar $q = 0.9^L$ is not the new semantic vector. The following entries separate those uses.

| New architecture | Meaning |
|---|---|
| $q$; $w$ | Semantic latent decoded into an answer; persistent workspace that changes its search geometry. |
| $E$; $g$; $H$ | Learned scalar energy; gradient $\nabla_q E$; Hessian $\nabla_q^2 E$. The historical TRM symbol $H$ instead counts outer cycles. |
| $M$; $m$, $\bar{m}$ | Positive-definite preconditioner; uniform lower/upper eigenvalue bounds. |
| $\alpha$; $\beta$; $\mu$ | Step size; gradient smoothness constant; PL or strong-convexity constant as specified. |
| $B$; $\mathcal{W}$ | Bounded workspace update; its invariant domain. Historical TRM $B$ instead denotes its answer-only update. |
| $R$; $D$ | Real-valued logit readout; discrete decoder, typically tokenwise argmax. |
| $\mathcal{M}$; $V$ | Intended attracting set; nonnegative potential measuring deviation from consistency. |
| $c$; $J_c$; $P$ | Learned constraint residual; its state Jacobian; tangent-nullspace projector. |
| $K$; $T$ | Dissipative matrix and energy-orthogonal velocity in the continuous attracting-set model. |
| Lyapunov function | A scalar progress measure non-increasing along the specified dynamics. |
| PL inequality | Gradient magnitude controls energy error; not automatically a pointwise state-distance bound. |
| Tangent / normal | Directions along a smooth manifold / perpendicular to it. |
| Gauge orbit / quotient | States related by a specified symmetry / the space after identifying those states. |

### Historical TRM notation and shared experimental terms

| Term | Meaning here |
|---|---|
| $h$, historical answer state | The tensor called `z_H`; its readout predicts the board. |
| $w$, historical working state | The tensor called `z_L`; affects future $h$ but is not directly decoded. |
| $x$ | Fixed puzzle embedding/injection for one continued computation. |
| $G$ | One shared two-block neural module used for both state updates. |
| $L$; $H$ | Working calls per outer update; outer updates per native macro. Native values are 6 and 3. |
| $A$; $B$; $F_L$ | Working-only map; answer-only map; outer map $B \circ A^L$. |

| Term | Meaning here |
|---|---|
| Macro-step | Three outer updates in the original Sudoku configuration. This is one logged pilot step. |
| Dense readout | Read after every answer update, including inside a macro. |
| Full carry / phase | Both stored states and where execution is within the update schedule. |
| Retention / loss | A correct answer stays correct / later becomes wrong, under a specified observation policy and window. |
| Recovery | A previously wrong answer becomes correct. Different from retention. |
| Logit / margin | A prediction score / the score gap between a chosen token and alternatives. Neither proves correctness. |
| $Q$ head | Model output trained for halting-related decisions; not an oracle of correctness. |
| Residual | Size of the change proposed by an update, e.g. $\|F(s) - s\|$. A small residual is not a correctness certificate. The learned constraint residual $c$ in Appendix A is a different object. |
| Fixed point | An exact state unchanged by the specified update map. Not necessarily unique, attracting or correct. |
| Attraction / basin | Nearby states approach a set; the collection of starting states that approach it. |
| Jacobian | Matrix describing the update's first-order response to small state changes. |
| Spectral radius | Largest eigenvalue magnitude; relevant to asymptotic local linearized stability. |
| Singular value | Measures norm gain for a linear map; the largest measures maximal one-update amplification. |
| Transient amplification | An error grows temporarily even though eventual linearized behavior may decay. |
| Damping | Accepting a fraction $\eta$ of a proposed update. |
| Paired comparison | Comparing different continuations of the same puzzles/states. |
| Percentage point (pp) | Absolute percentage difference: 78% to 80% is +2 pp. |
| Dev / held-out | Data for developing the idea / data reserved from that development; public-source reuse still matters. |

# References

[1] A. Jolicoeur-Martineau. *Less is More: Recursive Reasoning with Tiny Networks*. 2025. `https://arxiv.org/abs/2510.04871`. Official code: `https://github.com/SamsungSAILMontreal/TinyRecursiveModels`.

[2] S. Movahedi et al. *Fixed-Point Reasoners: Stable and Adaptive Deep Looped Transformers*. 2026. `https://arxiv.org/abs/2606.18206v1`. See also Appendix F on allocation.

[3] P. Li et al. *Scaling to Tens of Thousands of Test-Time Iterations with Loop-Native Attention Residuals* (InfiLoop). Version 2, 2026. `https://arxiv.org/abs/2610.11570v2`. Inspected code revision: `63716a9c46723d3592c8ad4fc5819855f22aaa47`.

[4] H. Krishna et al. *When Recursive Models Finish Computing*. 2026. `https://arxiv.org/abs/2609.26487v1`.

[5] B. Wang et al. *Scheduling Recursive Reasoning in Looped Transformers* (TAPS). 2026. `https://arxiv.org/abs/2609.36653v1`. Method definition, Section 5.3 and Appendix C.4 reviewed; comparator code not yet audited.

[6] S. Park and T. K. Rusch. *Looped Reasoning Finishes Earlier Than You Think* (QRR). 2026. `https://arxiv.org/abs/2610.18174`.

[7] Y. Inui, T. Konishi and Y. Kawahara. *Invariant-Measure Reasoners: Stable Representations for Latent Reasoning* (ImR). 2026. `https://arxiv.org/abs/2610.10996`. Primary abstract reviewed; detailed comparison pending.

[8] Readout Feedback (RoFB). 2026. `https://arxiv.org/html/2608.24136v2`. Primary introduction and method overview reviewed.

[9] Input-Conditioned Search (ICS). 2026. `https://arxiv.org/abs/2610.18547v2`. Primary abstract reviewed; detailed method/theory review pending.

[10] *Learning to Compute Is Not Learning to Retain or Reuse*. 2026. `https://arxiv.org/abs/2610.19768`.

[11] *Controlling Transient Amplification Improves Long-horizon Rollouts*. 2026. `https://arxiv.org/abs/2605.08856`.

[12] T. Spenke et al. *On the number of subproblem iterations per coupling step in partitioned fluid–structure interaction simulations*. 2023. `https://arxiv.org/abs/2303.08513`. Numerical-analysis precedent for coupled iteration, not evidence about TRM.

[13] Sheehan et al. Iteration error and stability in coupled PDE computations. `https://doi.org/10.1155/2015/787198`.

[14] W. Wang and J.-J. E. Slotine. Partial contraction and synchronization. Primary paper: `https://people.csail.mit.edu/paulina/jclub/partial-contraction.pdf`.

[15] I. R. Manchester and J.-J. E. Slotine. Transverse contraction and orbital stability. `https://arxiv.org/abs/1209.4433`.

[16] Output contraction for discrete-time nonlinear systems and recurrent neural networks. Source record: `https://pubmed.ncbi.nlm.nih.gov/40669402/`. Check the theorem's assumptions before applying it to a new architecture.

[17] L. N. Trefethen et al. *Hydrodynamic Stability Without Eigenvalues*. Primary paper: `https://people.maths.ox.ac.uk/trefethen/ttrd.pdf`.

[18] W. X. Chen, A. Gladstone, Z. M. Li, H. Ji and Y. Du. *Energy-Based Reasoners: Your Recurrent Reasoner Should Be an Energy-Based Model*. 2026. `https://arxiv.org/abs/2611.08977`. Full-text method reviewed; no reproduction claimed.

[19] Y. Du, S. Li, J. B. Tenenbaum and I. Mordatch. *Learning Iterative Reasoning through Energy Minimization*. ICML 2022. `https://energy-based-model.github.io/iterative-reasoning-as-energy-minimization/`.

[20] M. Eliasof, E. Haber and C. B. Schönlieb. *Graph Neural Dynamics via Learned Energy and Tangential Flows* (TANGO). ICML 2026. Primary proceedings entry: `https://proceedings.mlr.press/v306/eliasof26a.html`. Primary description reviewed; detailed theorem/code audit pending.

[21] G. Lifshitz, S. Zuler, O. Fouks and D. Raviv. *L-SR1: Learned Symmetric-Rank-One Preconditioning*. ICML 2026. Primary proceedings entry: `https://proceedings.mlr.press/v306/lifshitz26a.html`. Learned preconditioning comparator; primary description reviewed.

[22] A. White et al. *Projected Neural Differential Equations for Learning Constrained Dynamics*. 2024; revised 2026. `https://arxiv.org/abs/2410.23667`. Constraint-preserving differential-model reference; no direct reasoning-performance claim.

[23] P. Morris. *The Breakout Method for Escaping from Local Minima*. AAAI 1993, pp. 40–45. `https://cdn.aaai.org/AAAI/1993/AAAI93-007.pdf`.

[24] M. Ercsey-Ravasz and Z. Toroczkai. *Optimization hardness as transient chaos in an analog approach to constraint satisfaction*. Nature Physics 7, 966–970, 2011. `https://doi.org/10.1038/nphys2105`. Adaptive constraint-weight precedent; no digital-complexity improvement inferred.

[25] G. Raskutti and M. W. Mahoney. *Statistical and Algorithmic Perspectives on Randomized Sketching for Ordinary Least-Squares*. ICML 2015. `https://proceedings.mlr.press/v37/raskutti15.html`. General sketching context, not an identical nonlinear solver.

[26] R. Yuan, A. Lazaric and R. M. Gower. *Sketched Newton–Raphson*. SIAM Journal on Optimization, 2022. `https://arxiv.org/abs/2006.12120v5`. Section 3 equations (3.1)–(3.5) inspected directly for the residual-metric update and zero-residual interpolation property.

[27] Y. He and S. Wu. *The Lagrangian Brain: A Constraint Mechanics of Mind and Machine*. 2026. `https://arxiv.org/abs/2611.10737v1`. Appendix F.3 update description inspected; headline performance not reproduced.